In the above example, the outputs are present only at the times the transitions are taken. We can also generalize FSMs to support continuous-time outputs, but to do this, we need the notion of state refinements.

4.1.3 State Refinements

A hybrid system associates with each state of an FSM a dynamic behavior. Our first (very simple) example uses this capability merely to produce continuous-time outputs.

> **Example 4.2:** Suppose that instead of discrete outputs as in Example 4.1 we wish to produce a control signal whose value is 1 when the heat is on and 0 when the heat is off. Such a control signal could directly drive a heater. The thermostat in Figure 4.3 does this. In that figure, each state has a refinement that gives the value of the output h while the state machine is in that state.

In a hybrid system, the current state of the state machine has a **state refinement** that gives the dynamic behavior of the output as a function of the input. In the above simple example, the output is constant in each state, which is rather trivial dynamics. Hybrid systems can get much more elaborate.

The general structure of a hybrid system model is shown in Figure 4.4. In that figure, there is a two-state finite-state machine. Each state is associated with a state refinement

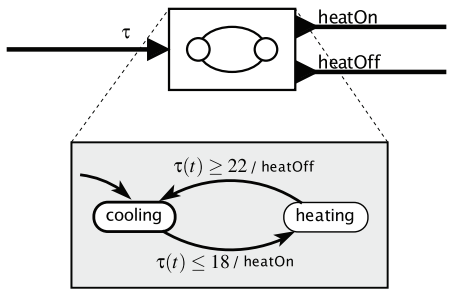


Figure 4.2: A thermostat modeled as an FSM with a continuous-time input signal.

labeled in the figure as a "time-based system." The state refinement defines dynamic behavior of the outputs and (possibly) additional continuous state variables. In addition, each transition can optionally specify set actions, which set the values of such additional state variables when a transition is taken. The example of Figure 4.3 is rather trivial, in that it has no continuous state variables, no output actions, and no set actions.

A hybrid system is sometimes called a **modal model** because it has a finite number of **mode**s, one for each state of the FSM, and when it is in a mode, it has dynamics specified by the state refinement. The states of the FSM may be referred to as modes rather than states, which as we will see, helps prevent confusion with states of the refinements.

The next simplest such dynamics, besides the rather trivial constant outputs of Example 4.2 is found in timed automata, which we discuss next.

4.2 Classes of Hybrid Systems

Hybrid systems can be quite elaborate. In this section, we first describe a relatively simple form known as timed automata. We then illustrate more elaborate forms that model nontrivial physical dynamics and nontrivial control systems.

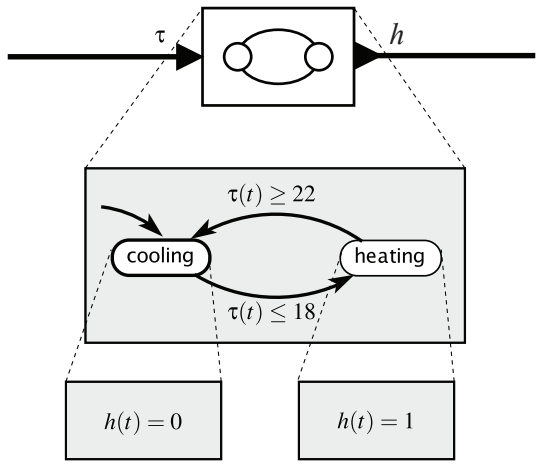


Figure 4.3: A thermostat with continuous-time output.

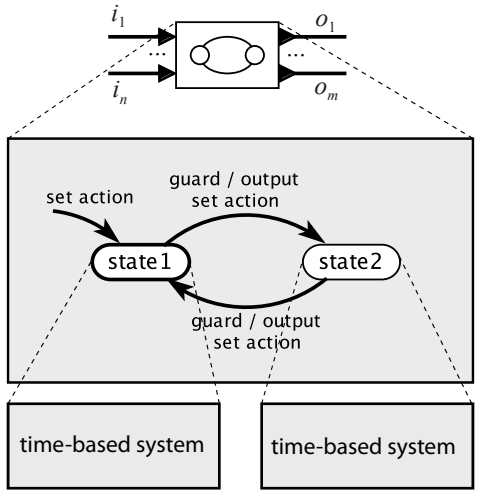


Figure 4.4: Notation for hybrid systems.

4.2.1 Timed Automata

Most cyber-physical systems require measuring the passage of time and performing actions at specific times. A device that measures the passage of time, a **clock**, has a particularly simple dynamics: its state progresses linearly in time. In this section, we describe **timed automata**, a formalism introduced by Alur and Dill (1994), which enable the construction of more complicated systems from such simple clocks.

Timed automata are the simplest non-trivial hybrid systems. They are modal models where the time-based refinements have very simple dynamics; all they do is measure the passage of time. A clock is modeled by a first-order differential equation,

$$\forall\, t \in T_m, \quad \dot{s}(t) = a,$$

where $s\colon \mathbb{R} \to \mathbb{R}$ is a continuous-time signal, $s(t)$ is the value of the clock at time t, and $T_m \subset \mathbb{R}$ is the subset of time during which the hybrid system is in mode m. The rate of the clock, a, is a constant while the system is in this mode.[1]

[1]The variant of timed automata we describe in this chapter differs from the original model of Alur and Dill (1994) in that the rates of clocks in different modes can be different. This variant is sometimes described in the literature as *multi-rate* timed automata.

Example 4.3: Recall the thermostat of Example 4.1, which uses hysteresis to prevent chattering. An alternative implementation that would also prevent chattering would use a single temperature threshold, but instead would require that the heater remain on or off for at least a minimum amount of time, regardless of the temperature. This design would not have the hysteresis property, but may be useful nonetheless. This can be modeled as a timed automaton as shown in Figure 4.5. In that figure, each state refinement has a clock, which is a continuous-time signal s with dynamics given by

$$\dot{s}(t) = 1 \; .$$

The value $s(t)$ increases linearly with t. Note that in that figure, the state refinement is shown directly with the name of the state in the state bubble. This shorthand is convenient when the refinement is relatively simple.

Notice that the initial state cooling has a set action on the dangling transition indicating the initial state, written as

$$s(t) := T_c \; .$$

As we did with extended state machines, we use the notation ":=" to emphasize that this is an assignment, not a predicate. This action ensures that when the thermostat starts, it can immediately transition to the heating mode if the temperature $\tau(t)$ is less than or equal to 20 degrees. The other two transitions each have set actions that reset the clock s to zero. The portion of the guard that specifies $s(t) \geq T_h$ ensures that the heater will always be on for at least time T_h. The portion of the guard that specifies $s(t) \geq T_c$ specifies that once the heater goes off, it will remain off for at least time T_c.

A possible execution of this timed automaton is shown in Figure 4.6. In that figure, we assume that the temperature is initially above the setpoint of 20 degrees, so the FSM remains in the cooling state until the temperature drops to 20 degrees. At that time t_1, it can take the transition immediately because $s(t_1) > T_c$. The transition resets s to zero and turns on the heater. The heater will remain on until time $t_1 + T_h$, assuming that the temperature only rises when the heater is on. At time $t_1 + T_h$, it will transition back to the cooling state and turn the heater off. (We assume here that a transition is taken as soon as it is enabled. Other transition semantics are possible.) It will cool until at least time T_c elapses and until

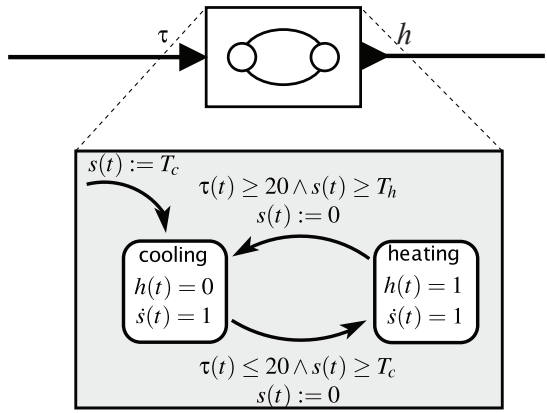


Figure 4.5: A timed automaton modeling a thermostat with a single temperature threshold, 20, and minimum times T_c and T_h in each mode.

the temperature drops again to 20 degrees, at which point it will turn the heater back on.

In the previous example the state of the system at any time t is not only the mode, heating or cooling, but also the current value $s(t)$ of the clock. We call s a **continuous state** variable, whereas heating and cooling are **discrete state**s. Thus, note that the term "state" for such a hybrid system can become confusing. The FSM has states, but so do the refinement systems (unless they are memoryless). When there is any possibility of confusion we explicitly refer to the states of the machine as modes.

Transitions between modes have actions associated with them. Sometimes, it is useful to have transitions from one mode back to itself, just so that the action can be realized. This is illustrated in the next example, which also shows a timed automaton that produces a pure output.

Example 4.4: The timed automaton in Figure 4.7 produces a pure output that will be present every T time units, starting at the time when the system begins

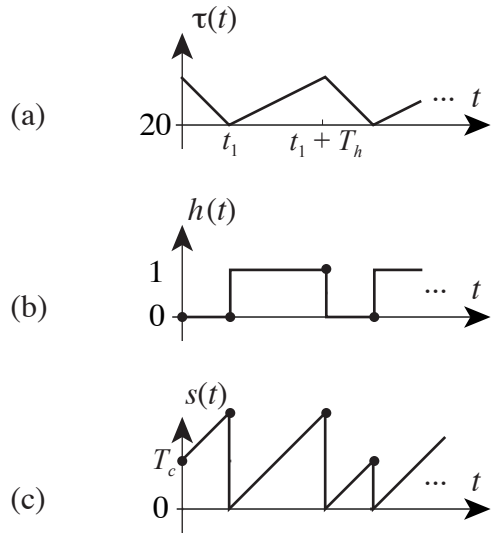


Figure 4.6: (a) A temperature input to the hybrid system of Figure 4.5, (b) the output h, and (c) the refinement state s.

executing. Notice that the guard on the transition, $s(t) \geq T$, is followed by an output action, *tick*, and a set action, $s(t) := 0$.

Figure 4.7 shows another notational shorthand that works well for simple diagrams. The automaton is shown directly inside the icon for its actor model.

Example 4.5: The traffic light controller of Figure 3.10 is a time triggered machine that assumes it reacts once each second. Figure 4.8 shows a timed automaton with the same behavior. It is more explicit about the passage of time in that its temporal dynamics do not depend on unstated assumptions about when the machine will react.

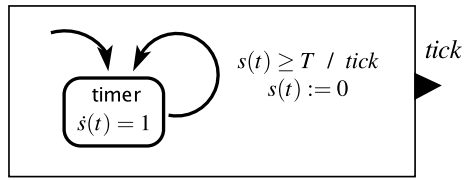


Figure 4.7: A timed automaton that generates a pure output event every T time units.

continuous variable: $x(t)$: $\mathbb{R}$
inputs: *pedestrian*: pure
outputs: *sigR*, *sigG*, *sigY*: pure

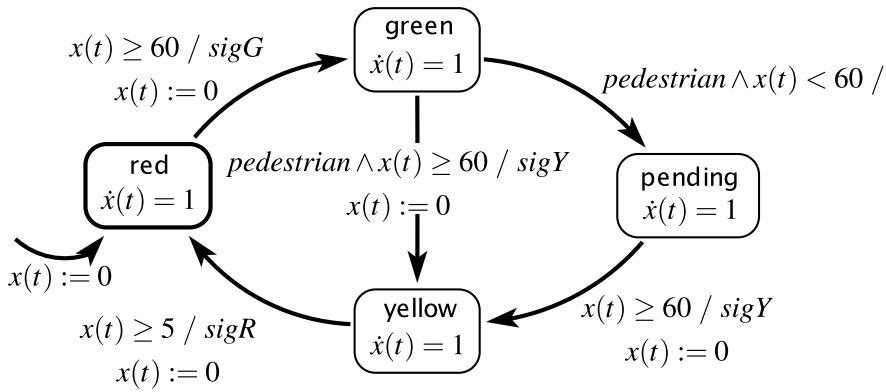


Figure 4.8: A timed automaton variant of the traffic light controller of Figure 3.10.

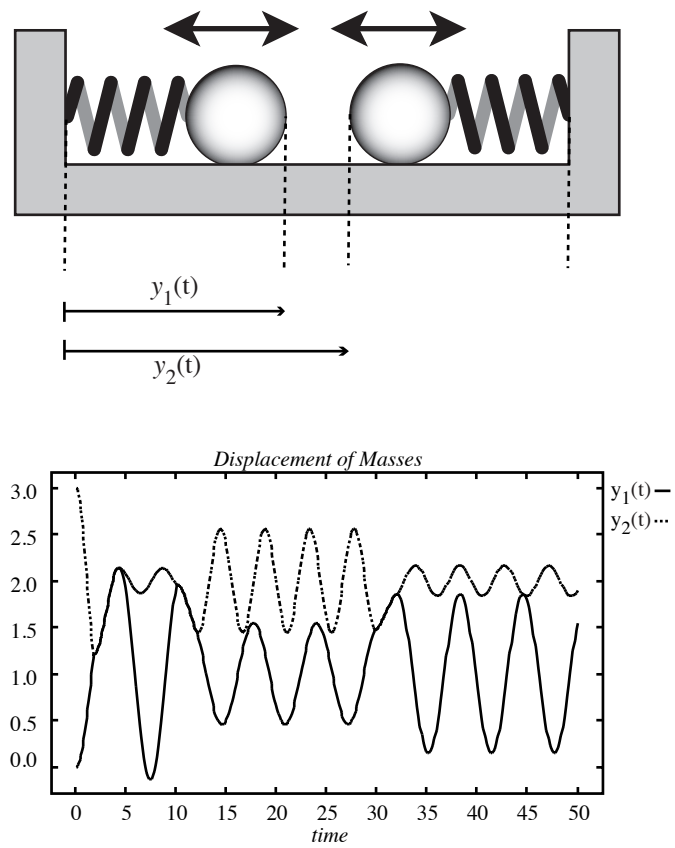


Figure 4.9: Sticky masses system considered in Example 4.6.

4.2.2 Higher-Order Dynamics

In timed automata, all that happens in the time-based refinement systems is that time passes. Hybrid systems, however, are much more interesting when the behavior of the refinements is more complex. Specifically,

Example 4.6: Consider the physical system depicted in Figure 4.9. Two sticky round masses are attached to springs. The springs are compressed or extended

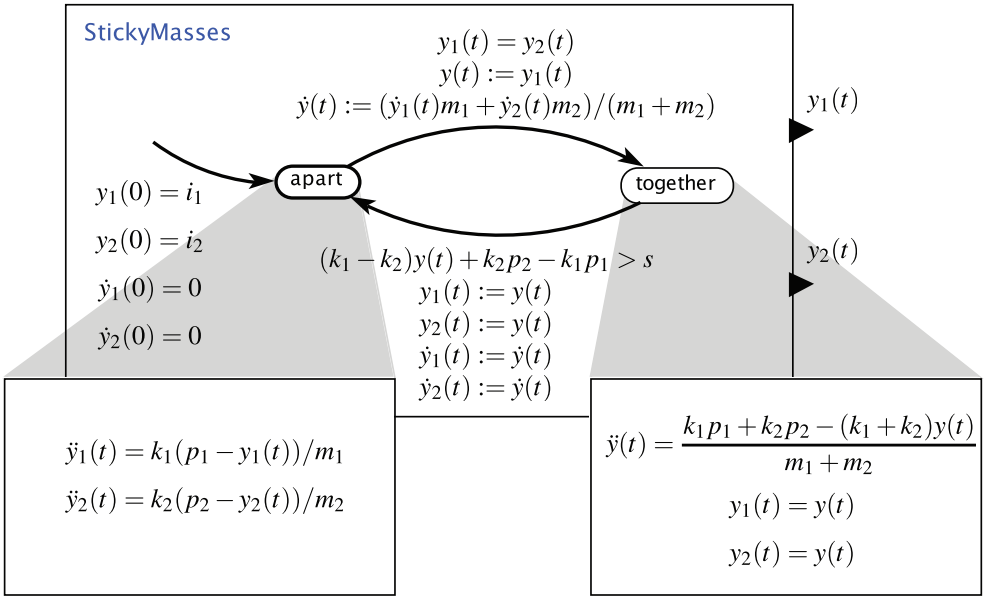


Figure 4.10: Hybrid system model for the sticky masses system considered in Example 4.6.

and then released. The masses oscillate on a frictionless table. If they collide, they stick together and oscillate together. After some time, the stickiness decays, and masses pull apart again.

A plot of the displacement of the two masses as a function of time is shown in Figure 4.9. Both springs begin compressed, so the masses begin moving towards one another. They almost immediately collide, and then oscillate together for a brief period until they pull apart. In this plot, they collide two more times, and almost collide a third time.

The physics of this problem is quite simple if we assume idealized springs. Let $y_1(t)$ denote the right edge of the left mass at time t, and $y_2(t)$ denote the left edge of the right mass at time t, as shown in Figure 4.9. Let p_1 and p_2 denote the neutral positions of the two masses, i.e., when the springs are neither extended nor compressed, so the force is zero. For an ideal spring, the force at time t on

the mass is proportional to $p_1 - y_1(t)$ (for the left mass) and $p_2 - y_2(t)$ (for the right mass). The force is positive to the right and negative to the left.

Let the spring constants be k_1 and k_2, respectively. Then the force on the left spring is $k_1(p_1 - y_1(t))$, and the force on the right spring is $k_2(p_2 - y_2(t))$. Let the masses be m_1 and m_2 respectively. Now we can use Newton's second law, which relates force, mass, and acceleration,

$$f = ma.$$

The acceleration is the second derivative of the position with respect to time, which we write $\ddot{y_1}(t)$ and $\ddot{y_2}(t)$. Thus, as long as the masses are separate, their dynamics are given by

$$\ddot{y}_1(t) = k_1(p_1 - y_1(t))/m_1 \tag{4.1}$$
$$\ddot{y}_2(t) = k_2(p_2 - y_2(t))/m_2. \tag{4.2}$$

When the masses collide, however, the situation changes. With the masses stuck together, they behave as a single object with mass $m_1 + m_2$. This single object is pulled in opposite directions by two springs. While the masses are stuck together, $y_1(t) = y_2(t)$. Let

$$y(t) = y_1(t) = y_2(t).$$

The dynamics are then given by

$$\ddot{y}(t) = \frac{k_1 p_1 + k_2 p_2 - (k_1 + k_2) y(t)}{m_1 + m_2}. \tag{4.3}$$

It is easy to see now how to construct a hybrid systems model for this physical system. The model is shown in Figure 4.10. It has two modes, apart and together. The refinement of the apart mode is given by (4.1) and (4.2), while the refinement of the together mode is given by (4.3).

We still have work to do, however, to label the transitions. The initial transition is shown in Figure 4.10 entering the apart mode. Thus, we are assuming the masses begin apart. Moreover, this transition is labeled with a set action that sets the initial positions of the two masses to i_1 and i_2 and the initial velocities to zero.

The transition from apart to together has the guard

$$y_1(t) = y_2(t)\ .$$

This transition has a set action which assigns values to two continuous state variables $y(t)$ and $\dot{y}(t)$, which will represent the motion of the two masses stuck together. The value it assigns to $\dot{y}(t)$ conserves momentum. The momentum of the left mass is $\dot{y}_1(t)m_1$, the momentum of the right mass is $\dot{y}_2(t)m_2$, and the momentum of the combined masses is $\dot{y}(t)(m_1 + m_2)$. To make these equal, it sets

$$\dot{y}(t) = \frac{\dot{y}_1(t)m_1 + \dot{y}_2(t)m_2}{m_1 + m_2}.$$

The refinement of the together mode gives the dynamics of y and simply sets $y_1(t) = y_2(t) = y(t)$, since the masses are moving together. The transition from apart to together sets $y(t)$ equal to $y_1(t)$ (it could equally well have chosen $y_2(t)$, since these are equal).

The transition from together to apart has the more complicated guard

$$(k_1 - k_2)y(t) + k_2p_2 - k_1p_1 > s,$$

where s represents the stickiness of the two masses. This guard is satisfied when the right-pulling force on the right mass exceeds the right-pulling force on the left mass by more than the stickiness. The right-pulling force on the right mass is simply

$$f_2(t) = k_2(p_2 - y(t))$$

and the right-pulling force on the left mass is

$$f_1(t) = k_1(p_1 - y(t)).$$

Thus,

$$f_2(t) - f_1(t) = (k_1 - k_2)y(t) + k_2p_2 - k_1p_1.$$

When this exceeds the stickiness s, then the masses pull apart.

An interesting elaboration on this example, considered in problem 11, modifies the together mode so that the stickiness is initialized to a starting value, but then decays according to the differential equation

$$\dot{s}(t) = -as(t)$$

where $s(t)$ is the stickiness at time t, and a is some positive constant. In fact, it is the dynamics of such an elaboration that is plotted in Figure 4.9.

As in Example 4.4, it is sometimes useful to have hybrid system models with only one state. The actions on one or more state transitions define the discrete event behavior that combines with the time-based behavior.

Example 4.7: Consider a bouncing ball. At time $t = 0$, the ball is dropped from a height $y(0) = h_0$, where h_0 is the initial height in meters. It falls freely. At some later time t_1 it hits the ground with a velocity $\dot{y}(t_1) < 0$ m/s (meters per second). A *bump* event is produced when the ball hits the ground. The collision is **inelastic** (meaning that kinetic energy is lost), and the ball bounces back up with velocity $-a\dot{y}(t_1)$, where a is constant with $0 < a < 1$. The ball will then rise to a certain height and fall back to the ground repeatedly.

The behavior of the bouncing ball can be described by the hybrid system of Figure 4.11. There is only one mode, called free. When it is not in contact with the ground, we know that the ball follows the second-order differential equation,

$$\ddot{y}(t) = -g, \tag{4.4}$$

where $g = 9.81$ m/sec^2 is the acceleration imposed by gravity. The continuous state variables of the free mode are

$$s(t) = \begin{bmatrix} y(t) \\ \dot{y}(t) \end{bmatrix}$$

with the initial conditions $y(0) = h_0$ and $\dot{y}(0) = 0$. It is then a simple matter to rewrite (4.4) as a first-order differential equation,

$$\dot{s}(t) = f(s(t)) \tag{4.5}$$

for a suitably chosen function f.

At the time $t = t_1$ when the ball first hits the ground, the guard

$$y(t) = 0$$

is satisfied, and the self-loop transition is taken. The output *bump* is produced, and the set action $\dot{y}(t) := -a\dot{y}(t)$ changes $\dot{y}(t_1)$ to have value $-a\dot{y}(t_1)$. Then (4.4) is followed again until the guard becomes true again.

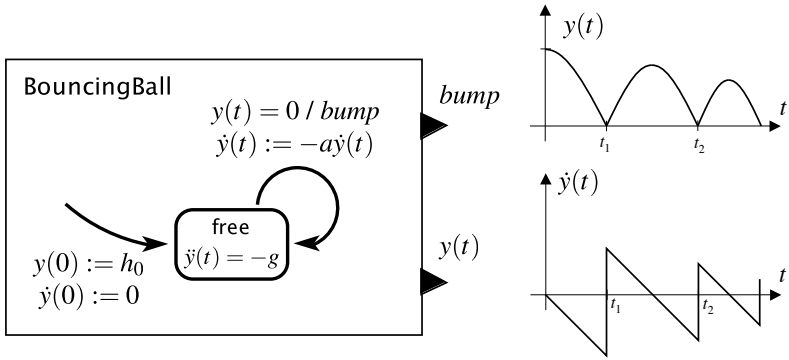


Figure 4.11: The motion of a bouncing ball may be described as a hybrid system with only one mode. The system outputs a *bump* each time the ball hits the ground, and also outputs the position of the ball. The position and velocity are plotted versus time at the right.

By integrating (4.4) we get, for all $t \in (0, t_1)$,

$$\begin{aligned} \dot{y}(t) &= -gt, \\ y(t) &= y(0) + \int_0^t \dot{y}(\tau)d\tau = h_0 - \frac{1}{2}gt^2. \end{aligned}$$

So $t_1 > 0$ is determined by $y(t_1) = 0$. It is the solution to the equation

$$h_0 - \frac{1}{2}gt^2 = 0.$$

Thus,

$$t_1 = \sqrt{2h_0/g}.$$

Figure 4.11 plots the continuous state versus time.

The bouncing ball example above has an interesting difficulty that is explored in Exercise 10. Specifically, the time between bounces gets smaller as time increases. In fact, it

gets smaller fast enough that an infinite number of bounces occur in a finite amount of time. A system with an infinite number of discrete events in a finite amount of time is called a **Zeno** system, after Zeno of Elea, a pre-Socratic Greek philosopher famous for his paradoxes. In the physical world, of course, the ball will eventually stop bouncing; the Zeno behavior is an artifact of the model. Another example of a Zeno hybrid system is considered in Exercise 13.

4.2.3 Supervisory Control

A control system involves four components: a system called the **plant**, the physical process that is to be controlled; the environment in which the plant operates; the sensors that measure some variables of the plant and the environment; and the controller that determines the mode transition structure and selects the time-based inputs to the plant. The controller has two levels: the **supervisory control** that determines the mode transition structure, and the **low-level control** that determines the time-based inputs to the plant. Intuitively, the supervisory controller determines which of several strategies should be followed, and the low-level controller implements the selected strategy. Hybrid systems are ideal for modeling such two-level controllers. We show how through a detailed example.

Example 4.8: Consider an **automated guided vehicle** (**AGV**) that moves along a closed track painted on a warehouse or factory floor. We will design a controller so that the vehicle closely follows the track.

The vehicle has two degrees of freedom. At any time t, it can move forward along its body axis with speed $u(t)$ with the restriction that $0 \leq u(t) \leq 10$ mph (miles per hour). It can also rotate about its center of gravity with an angular speed $\omega(t)$ restricted to $-\pi \leq \omega(t) \leq \pi$ radians/second. We ignore the inertia of the vehicle, so we assume that we can instantaneously change the velocity or angular speed.

Let $(x(t), y(t)) \in \mathbb{R}^2$ be the position relative to some fixed coordinate frame and $\theta(t) \in (-\pi, \pi]$ be the angle (in radians) of the vehicle at time t, as shown in Figure 4.12. In terms of this coordinate frame, the motion of the vehicle is given

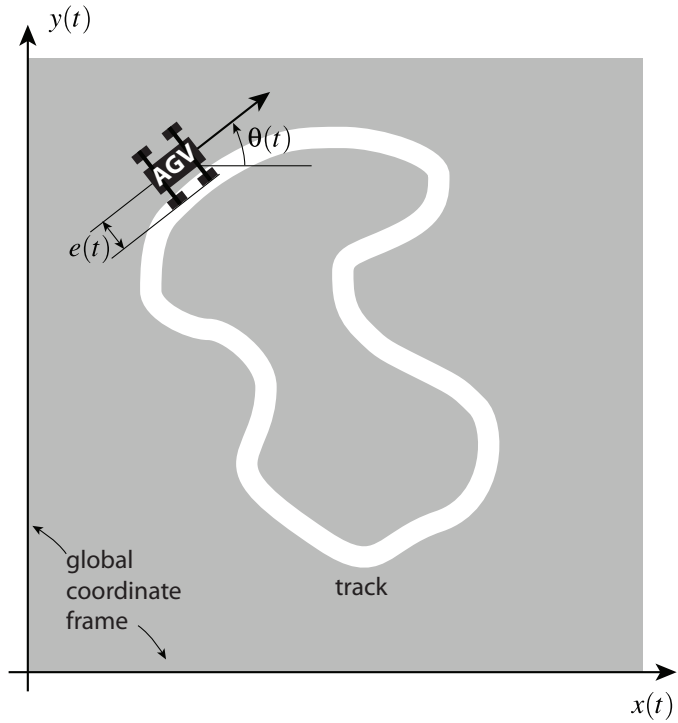


Figure 4.12: Illustration of the automated guided vehicle of Example 4.8. The vehicle is following a curved painted track, and has deviated from the track by a distance $e(t)$. The coordinates of the vehicle at time t with respect to the global coordinate frame are $(x(t), y(t), \theta(t))$.

by a system of three differential equations,

$$\begin{aligned} \dot{x}(t) &= u(t)\cos\theta(t), \\ \dot{y}(t) &= u(t)\sin\theta(t), \\ \dot{\theta}(t) &= \omega(t). \end{aligned} \tag{4.6}$$

Equations (4.6) describe the plant. The environment is the closed painted track. It could be described by an equation. We will describe it indirectly below by means of a sensor.

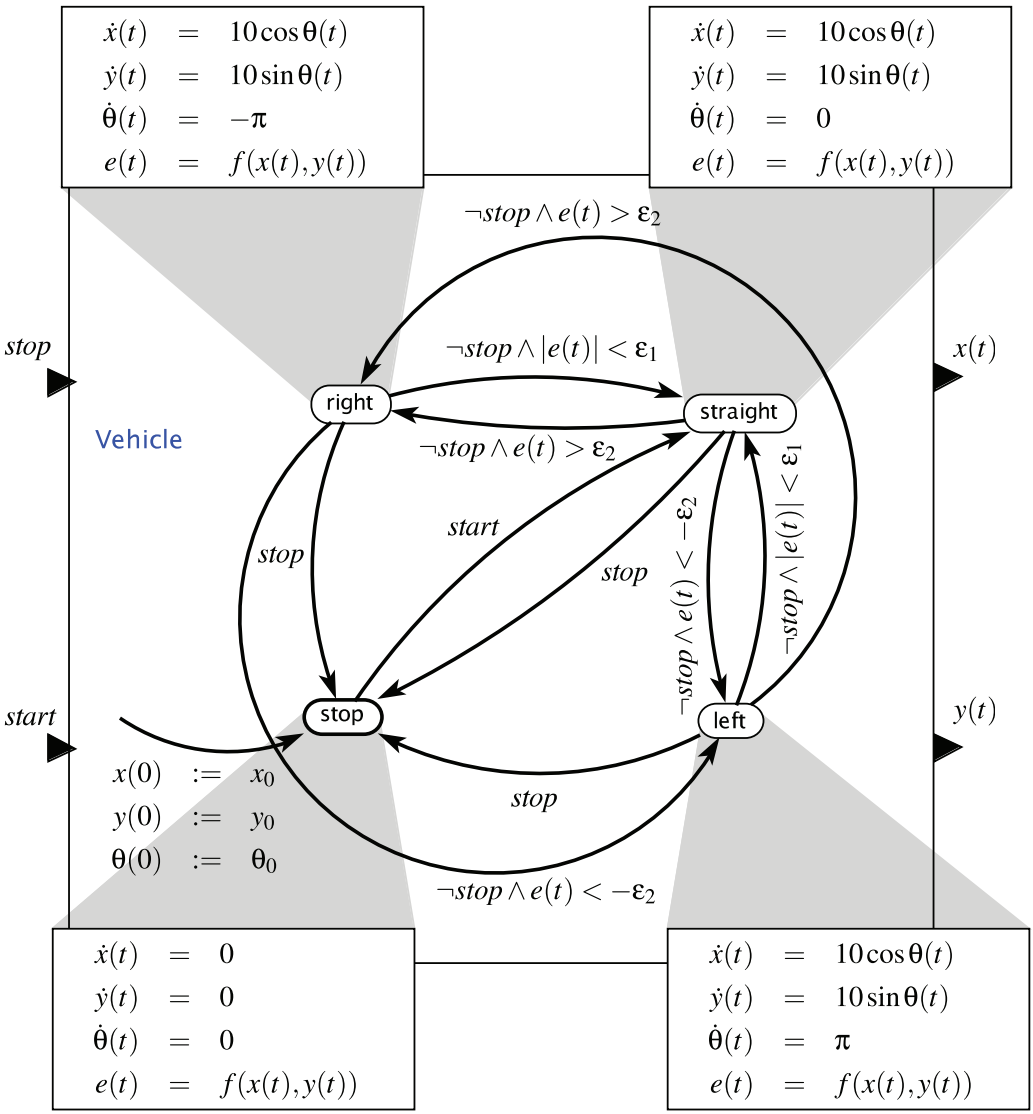


Figure 4.13: The automatic guided vehicle of Example 4.8 has four modes: stop, straight, left, right.

The two-level controller design is based on a simple idea. The vehicle always moves at its maximum speed of 10 mph. If the vehicle strays too far to the left of the track, the controller steers it towards the right; if it strays too far to the right of the track, the controller steers it towards the left. If the vehicle is close to the track, the controller maintains the vehicle in a straight direction. Thus the controller guides the vehicle in four modes, left, right, straight, and stop. In stop mode, the vehicle comes to a halt.

The following differential equations govern the AGV's motion in the refinements of the four modes. They describe the low-level controller, i.e., the selection of the time-based plant inputs in each mode.

straight

$$\begin{aligned} \dot{x}(t) &= 10\cos\theta(t) \\ \dot{y}(t) &= 10\sin\theta(t) \\ \dot{\theta}(t) &= 0 \end{aligned}$$

left

$$\begin{aligned} \dot{x}(t) &= 10\cos\theta(t) \\ \dot{y}(t) &= 10\sin\theta(t) \\ \dot{\theta}(t) &= \pi \end{aligned}$$

right

$$\begin{aligned} \dot{x}(t) &= 10\cos\theta(t) \\ \dot{y}(t) &= 10\sin\theta(t) \\ \dot{\theta}(t) &= -\pi \end{aligned}$$

stop

$$\begin{aligned} \dot{x}(t) &= 0 \\ \dot{y}(t) &= 0 \\ \dot{\theta}(t) &= 0 \end{aligned}$$

In the stop mode, the vehicle is stopped, so $x(t)$, $y(t)$, and $\theta(t)$ are constant. In the left mode, $\theta(t)$ increases at the rate of π radians/second, so from Figure 4.12

we see that the vehicle moves to the left. In the right mode, it moves to the right. In the straight mode, $\theta(t)$ is constant, and the vehicle moves straight ahead with a constant heading. The refinements of the four modes are shown in the boxes of Figure 4.13.

We design the supervisory control governing transitions between modes in such a way that the vehicle closely follows the track, using a sensor that determines how far the vehicle is to the left or right of the track. We can build such a sensor using photodiodes. Let's suppose the track is painted with a light-reflecting color, whereas the floor is relatively dark. Underneath the AGV we place an array of photodiodes as shown in Figure 4.14. The array is perpendicular to the AGV body axis. As the AGV passes over the track, the diode directly above the track generates more current than the other diodes. By comparing the magnitudes of the currents through the different diodes, the sensor estimates the displacement $e(t)$ of the center of the array (hence, the center of the AGV) from the track. We adopt the convention that $e(t) < 0$ means that the AGV is to the right of the track and $e(t) > 0$ means it is to the left. We model the sensor output as a function f of the AGV's position,

$$\forall t, \quad e(t) = f(x(t), y(t)).$$

The function f of course depends on the environment—the track. We now specify the supervisory controller precisely. We select two thresholds, $0 < \epsilon_1 < \epsilon_2$, as shown in Figure 4.14. If the magnitude of the displacement is small, $|e(t)| < \epsilon_1$, we consider that the AGV is close enough to the track, and the AGV can move straight ahead, in straight mode. If $e(t) > \epsilon_2$ ($e(t)$ is large and positive), the AGV has strayed too far to the left and must be steered to the right, by switching to right mode. If $e(t) < -\epsilon_2$ ($e(t)$ is large and negative), the AGV has strayed too far to the right and must be steered to the left, by switching to left mode. This control logic is captured in the mode transitions of Figure 4.13. The inputs are pure signals *stop* and *start*. These model an operator that can stop or start the AGV. There is no continuous-time input. The outputs represent the position of the vehicle, $x(t)$ and $y(t)$. The initial mode is stop, and the initial values of its refinement are (x_0, y_0, θ_0).

We analyze how the AGV will move. Figure 4.15 sketches one possible trajectory. Initially the vehicle is within distance ϵ_1 of the track, so it moves straight. At some

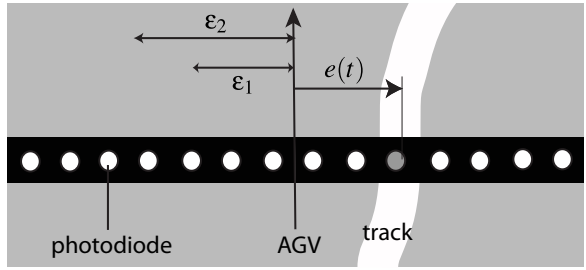


Figure 4.14: An array of photodiodes under the AGV is used to estimate the displacement e of the AGV relative to the track. The photodiode directly above the track generates more current.

later time, the vehicle goes too far to the left, so the guard

$$\neg stop \wedge e(t) > \epsilon_2$$

is satisfied, and there is a mode switch to right. After some time, the vehicle will again be close enough to the track, so the guard

$$\neg stop \wedge |e(t)| < \epsilon_1$$

is satisfied, and there is a mode switch to straight. Some time later, the vehicle is too far to the right, so the guard

$$\neg stop \wedge e(t) < -\epsilon_2$$

is satisfied, and there is a mode switch to left. And so on.

The example illustrates the four components of a control system. The plant is described by the differential equations (4.6) that govern the evolution of the continuous state at time t, $(x(t), y(t), \theta(t))$, in terms of the plant inputs u and ω. The second component is the environment—the closed track. The third component is the sensor, whose output at time t, $e(t) = f(x(t), y(t))$, gives the position of the AGV relative to the track. The fourth component is the two-level controller. The supervisory controller comprises the four modes and the guards that determine when to switch between modes. The low-level

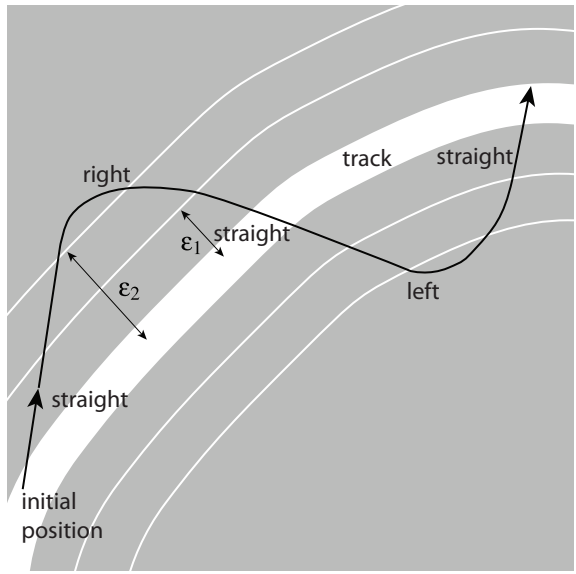


Figure 4.15: A trajectory of the AGV, annotated with modes.

controller specifies how the time-based inputs to the plant, u and ω, are selected in each mode.

4.3 Summary

Hybrid systems provide a bridge between time-based models and state-machine models. The combination of the two families of models provides a rich framework for describing real-world systems. There are two key ideas. First, discrete events (state changes in a state machine) get embedded in a time base. Second, a hierarchical description is particularly useful, where the system undergoes discrete transitions between different modes of operation. Associated with each mode of operation is a time-based system called the refinement of the mode. Mode transitions are taken when guards that specify the combination of inputs and continuous states are satisfied. The action associated with a transition, in turn, sets the continuous state in the destination mode.

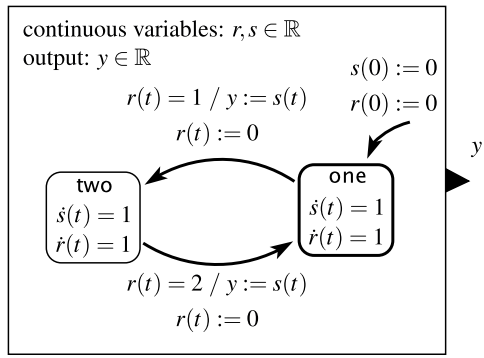
The behavior of a hybrid system is understood using the tools of state machine analysis for mode transitions and the tools of time-based analysis for the refinement systems. The design of hybrid systems similarly proceeds on two levels: state machines are designed to achieve the appropriate logic of mode transitions, and continuous refinement systems are designed to secure the desired time-based behavior in each mode.

Exercises

1. Construct (on paper is sufficient) a timed automaton similar to that of Figure 4.7 which produces *tick* at times $1, 2, 3, 5, 6, 7, 8, 10, 11, \cdots$. That is, ticks are produced with intervals between them of 1 second (three times) and 2 seconds (once).

2. The objective of this problem is to understand a timed automaton, and then to modify it as specified.

   (a) For the timed automaton shown below, describe the output y. Avoid imprecise or sloppy notation.

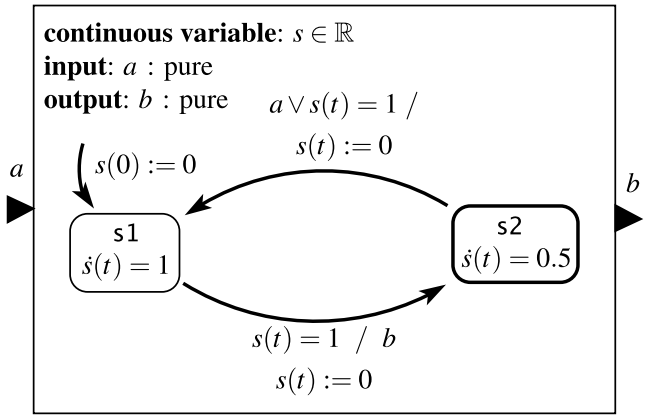


   (b) Assume there is a new pure input *reset*, and that when this input is present, the hybrid system starts over, behaving as if it were starting at time 0 again. Modify the hybrid system from part (a) to do this.

3. In Exercise 6 of Chapter 2, we considered a DC motor that is controlled by an input voltage. Controlling a motor by varying an input voltage, in reality, is often not practical. It requires analog circuits that are capable of handling considerable power. Instead, it is common to use a fixed voltage, but to turn it on and off periodically to vary the amount of power delivered to the motor. This technique is called pulse width modulation (PWM).

   Construct a timed automaton that provides the voltage input to the motor model from Exercise 6. Your hybrid system should assume that the PWM circuit delivers a 25 kHz square wave with a duty cycle between zero and 100%, inclusive. The input to your hybrid system should be the duty cycle, and the output should be the voltage.

4. Consider the following timed automaton:



Assume that the input signals a and b are discrete continuous-time signals, meaning that each can be given as a function of form $a: \mathbb{R} \to \{present, absent\}$, where at almost all times $t \in \mathbb{R}$, $a(t) = absent$. Assume that the state machine can take at most one transition at each distinct time t, and that machine begins executing at time $t = 0$.

(a) Sketch the output b if the input a is present only at times

$$t = 0.75, 1.5, 2.25, 3, 3.75, 4.5, \cdots$$

Include at least times from $t = 0$ to $t = 5$.

(b) Sketch the output b if the input a is present only at times $t = 0, 1, 2, 3, \cdots$.

(c) Assuming that the input a can be any discrete signal at all, find a lower bound on the amount of time between events b. What input signal a (if any) achieves this lower bound?

5. You have an analog source that produces a pure tone. You can switch the source on or off by the input event *on* or *off*. Construct a timed automaton that provides the *on* and *off* signals as outputs, to be connected to the inputs of the tone generator. Your system should behave as follows. Upon receiving an input event *ring*, it should produce an 80 ms-long sound consisting of three 20 ms-long bursts of the pure tone separated by two 10 ms intervals of silence. What does your system do if it receives two *ring* events that are 50 ms apart?

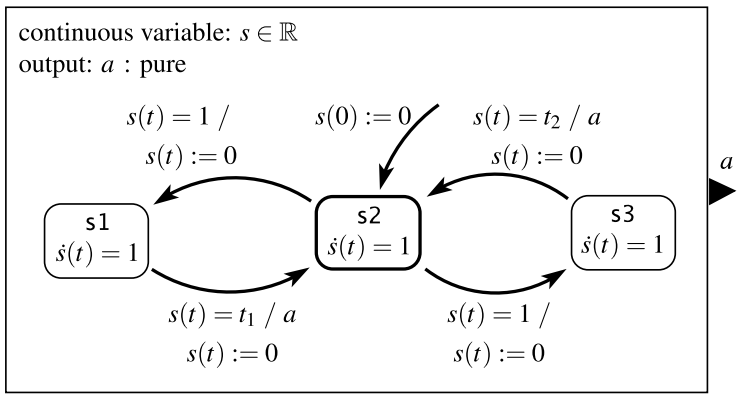
6. Automobiles today have the features listed below. Implement each feature as a timed automaton.

   (a) The dome light is turned on as soon as any door is opened. It stays on for 30 seconds after all doors are shut. What sensors are needed?

   (b) Once the engine is started, a beeper is sounded and a red light warning is indicated if there are passengers that have not buckled their seat belt. The beeper stops sounding after 30 seconds, or as soon the seat belts are buckled, whichever is sooner. The warning light is on all the time the seat belt is unbuckled. **Hint:** Assume the sensors provide a *warn* event when the ignition is turned on and there is a seat with passenger not buckled in, or if the ignition is already on and a passenger sits in a seat without buckling the seatbelt. Assume further that the sensors provide a *noWarn* event when a passenger departs from a seat, or when the buckle is buckled, or when the ignition is turned off.

7. A programmable thermostat allows you to select 4 times, $0 \leq T_1 \leq \cdots \leq T_4 < 24$ (for a 24-hour cycle) and the corresponding setpoint temperatures $a_1, \cdots, a_4$. Construct a timed automaton that sends the event a_i to the heating systems controller. The controller maintains the temperature close to the value a_i until it receives the next event. How many timers and modes do you need?

8. Consider the following timed automaton:



Assume t_1 and t_2 are positive real numbers. What is the minimum amount of time between events a? That is, what is the smallest possible time between two times when the signal a is present?

9. Figure 4.16 depicts the intersection of two one-way streets, called Main and Secondary. A light on each street controls its traffic. Each light goes through a cycle consisting of a red (R), green (G), and yellow (Y) phases. It is a safety requirement that when one light is in its green or yellow phase, the other is in its red phase. The yellow phase is always 5 seconds long.

   The traffic lights operate as follows. A sensor in the secondary road detects a vehicle. While no vehicle is detected, there is a 4 minute-long cycle with the main light having 3 minutes of green, 5 seconds of yellow, and 55 seconds of red. The secondary light is red for 3 minutes and 5 seconds (while the main light is green and yellow), green for 50 seconds, then yellow for 5 seconds.

   If a vehicle is detected on the secondary road, the traffic light quickly gives a right of way to the secondary road. When this happens, the main light aborts its green phase and immediately switches to its 5 second yellow phase. If the vehicle is detected while the main light is yellow or red, the system continues as if there were no vehicle.

   Design a hybrid system that controls the lights. Let this hybrid system have six pure outputs, one for each light, named *mG*, *mY*, and *mR*, to designate the main

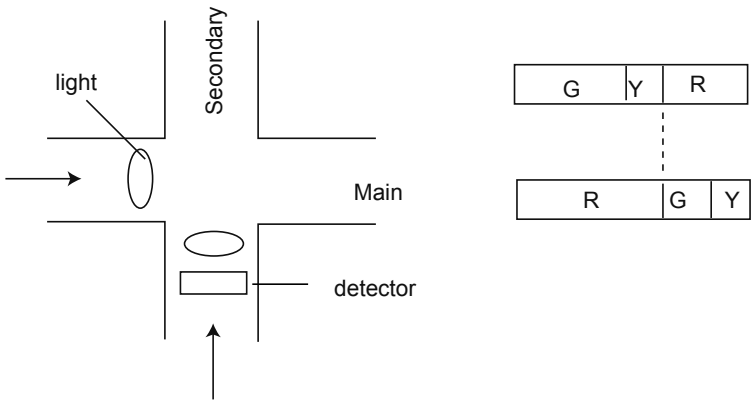


Figure 4.16: Traffic lights control the intersection of a main street and a secondary street. A detector senses when a vehicle crosses it. The red phase of one light must coincide with the green and yellow phases of the other light.

light being green, yellow, or red, respectively, and *sG*, *sY*, and *sR*, to designate the secondary light being green, yellow, or red, respectively. These signals should be generated to turn on a light. You can implicitly assume that when one light is turned on, whichever has been on is turned off.

10. For the bouncing ball of Example 4.7, let t_n be the time when the ball hits the ground for the n-th time, and let $v_n = \dot{y}(t_n)$ be the velocity at that time.

    (a) Find a relation between v_{n+1} and v_n for $n > 1$, and then calculate v_n in terms of v_1.

    (b) Obtain t_n in terms of v_1 and a. Use this to show that the bouncing ball is a Zeno system. **Hint:** The **geometric series identity** might be useful, where for $|b| < 1$,
    $$\sum_{m=0}^{\infty} b^m = \frac{1}{1-b}.$$

    (c) Calculate the maximum height reached by the ball after successive bumps.

11. Elaborate the hybrid system model of Figure 4.10 so that in the *together* mode, the stickiness decays according to the differential equation
    $$\dot{s}(t) = -as(t)$$
    where $s(t)$ is the stickiness at time t, and a is some positive constant. On the transition into this mode, the stickiness should be initialized to some starting stickiness b.

12. Show that the trajectory of the AGV of Figure 4.13 while it is in *left* or *right* mode is a circle. What is the radius of this circle, and how long does it take to complete a circle?

13. Consider Figure 4.17 depicting a system comprising two tanks containing water. Each tank is leaking at a constant rate. Water is added at a constant rate to the system through a hose, which at any point in time is filling either one tank or the other. It is assumed that the hose can switch between the tanks instantaneously. For $i \in \{1, 2\}$, let x_i denote the volume of water in Tank i and $v_i > 0$ denote the constant flow of water out of Tank i. Let w denote the constant flow of water into the system. The objective is to keep the water volumes above r_1 and r_2, respectively, assuming that the water volumes are above r_1 and r_2 initially. This is to be achieved

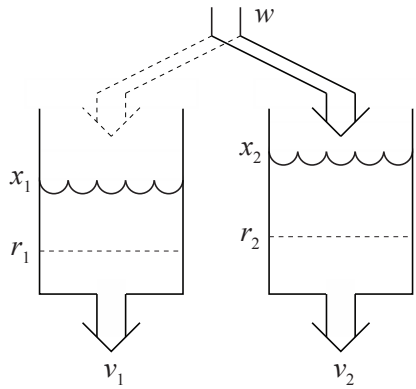


Figure 4.17: Water tank system.

by a controller that switches the inflow to Tank 1 whenever $x_1(t) \leq r_1(t)$ and to Tank 2 whenever $x_2(t) \leq r_2(t)$.

The hybrid automaton representing this two-tank system is given in Figure 4.18.

Answer the following questions:

(a) Construct a model of this hybrid automaton in Ptolemy II, LabVIEW, or Simulink. Use the following parameter values: $r_1 = r_2 = 0$, $v_1 = v_2 = 0.5$, and $w = 0.75$. Set the initial state to be $(q_1, (0, 1))$. (That is, initial value $x_1(0)$ is 0 and $x_2(0)$ is 1.)

Verify that this hybrid automaton is Zeno. What is the reason for this Zeno behavior? Simulate your model and plot how x_1 and x_2 vary as a function of time t, simulating long enough to illustrate the Zeno behavior.

(b) A Zeno system may be **regularized** by ensuring that the time between transitions is never less than some positive number ϵ. This can be emulated by inserting extra modes in which the hybrid automaton dwells for time ϵ. Use regularization to make your model from part (a) non-Zeno. Again, plot x_1 and x_2 for the same length of time as in the first part. State the value of ϵ that you used.

Include printouts of your plots with your answer.

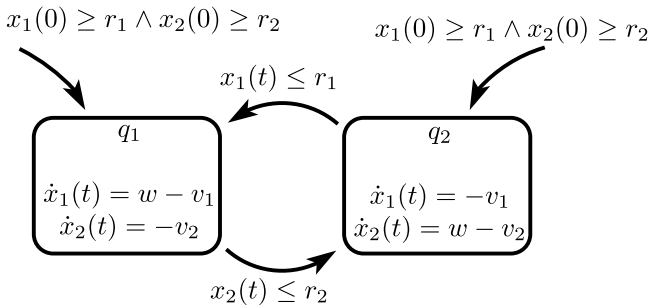


Figure 4.18: Hybrid automaton representing water tank system.

5 Composition of State Machines

5.1 Concurrent Composition 111
Sidebar: About Synchrony 112
5.1.1 Side-by-Side Synchronous Composition 113
5.1.2 Side-by-Side Asynchronous Composition 116
Sidebar: Scheduling Semantics for Asynchronous Composition 118
5.1.3 Shared Variables 119
5.1.4 Cascade Composition 122
5.1.5 General Composition 125
5.2 Hierarchical State Machines 126
5.3 Summary 130
Exercises 132

State machines provide a convenient way to model behaviors of systems. One disadvantage that they have is that for most interesting systems, the number of states is very large, often even infinite. Automated tools can handle large state spaces, but humans have more difficulty with any direct representation of a large state space.

A time-honored principle in engineering is that complicated systems should be described as compositions of simpler systems. This chapter gives a number of ways to do this with

state machines. The reader should be aware, however, that there are many subtly different ways to compose state machines. Compositions that look similar on the surface may mean different things to different people. The rules of notation of a model are called its **syntax**, and the meaning of the notation is called its **semantics**.

Example 5.1: In the standard syntax of arithmetic, a plus sign $+$ has a number or expression before it, and a number or expression after it. Hence, $1+2$, a sequence of three symbols, is a valid arithmetic expression, but $1+$ is not. The semantics of the expression $1 + 2$ is the addition of two numbers. This expression means "the number three, obtained by adding 1 and 2." The expression $2 + 1$ is syntactically different, but semantically identical (because addition is commutative).

The models in this book predominantly use a visual syntax, where the elements are boxes, circles, arrows, etc., rather than characters in a character set, and where the positioning of the elements is not constrained to be a sequence. Such syntaxes are less standardized than, for example, the syntax of arithmetic. We will see that the same syntax can have many different semantics, which can cause no end of confusion.

Example 5.2: A now popular notation for concurrent composition of state machines called Statecharts was introduced by Harel (1987). Although they are all based on the same original paper, many variants of Statecharts have evolved (von der Beeck, 1994). These variants often assign different semantics to the same syntax.

In this chapter, we assume an actor model for extended state machines using the syntax summarized in Figure 5.1. The semantics of a single such state machine is described in Chapter 3. This chapter will discuss the semantics that can be assigned to compositions of multiple such machines.

The first composition technique we consider is concurrent composition. Two or more state machines react either simultaneously or independently. If the reactions are simultaneous, we call it **synchronous composition**. If they are independent, then we call it

asynchronous composition. But even within these classes of composition, many subtle variations in the semantics are possible. These variations mostly revolve around whether and how the state machines communicate and share variables.

The second composition technique we will consider is hierarchy. Hierarchical state machines can also enable complicated systems to be described as compositions of simpler systems. Again, we will see that subtle differences in semantics are possible.

5.1 Concurrent Composition

To study concurrent composition of state machines, we will proceed through a sequence of patterns of composition. These patterns can be combined to build arbitrarily complicated systems. We begin with the simplest case, side-by-side composition, where the state machines being composed do not communicate. We then consider allowing communication through shared variables, showing that this creates significant subtleties that can complicate modeling. We then consider communication through ports, first looking at serial composition, then expanding to arbitrary interconnections. We consider both synchronous and asynchronous composition for each type of composition.

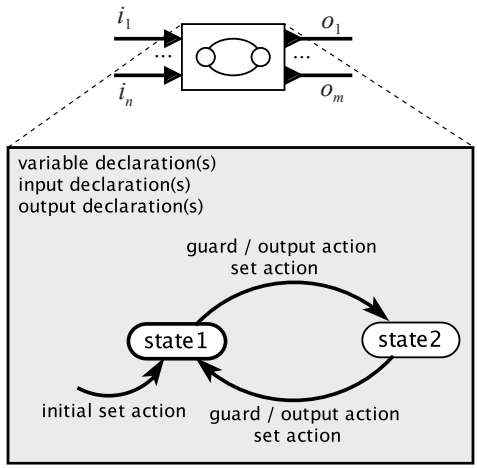


Figure 5.1: Summary of notation for state machines used in this chapter.

About Synchrony

The term **synchronous** means (1) occurring or existing at the same time or (2) moving or operating at the same rate. In engineering and computer science, the term has a number of meanings that are mostly consistent with these definitions, but oddly inconsistent with one another. In referring to concurrent software constructed using threads or processes, synchronous communication refers to a rendezvous style of communication, where the sender of a message must wait for the receiver to be ready to receive, and the receiver must wait for the sender. Conceptually, the two threads see the communication occurring at the same time, consistent with definition (1). In Java, the keyword `synchronized` defines blocks of code that are not permitted to execute simultaneously. Oddly, two code blocks that are synchronized *cannot* "occur" (execute) at the same time, which is inconsistent with both definitions.

In the world of software, there is yet a third meaning of the word synchronous, and it is this third meaning that we use in this chapter. This third meaning underlies the synchronous languages (see box on page 148). Two key ideas govern these languages. First, the outputs of components in a program are (conceptually) simultaneous with their inputs (this is called the **synchrony hypothesis**). Second, components in a program execute (conceptually) simultaneously and instantaneously. Real executions do not literally occur simultaneously nor instantaneously, and outputs are not really simultaneous with the inputs, but a correct execution must behave as if they were. This use of the word synchronous is consistent with *both* definitions above; executions of components occur at the same time and operate at the same rate.

In circuit design, the word synchronous refers to a style of design where a clock that is distributed throughout a circuit drives latches that record their inputs on edges of the clock. The time between clock edges needs to be sufficient for circuits between latches to settle. Conceptually, this model is similar to the model in synchronous languages. Assuming that the circuits between latches have zero delay is equivalent to the synchrony hypothesis, and global clock distribution gives simultaneous and instantaneous execution.

In power systems engineering, synchronous means that electrical waveforms have the same frequency and phase. In signal processing, synchronous means that signals have the same sample rate, or that their sample rates are fixed multiples of one another. The term synchronous dataflow, described in Section 6.3.2, is based on this latter meaning of the word synchronous. This usage is consistent with definition (2).

5.1.1 Side-by-Side Synchronous Composition

The first pattern of composition that we consider is **side-by-side composition**, illustrated for two actors in Figure 5.2. In this pattern, we assume that the inputs and outputs of the two actors are disjoint, i.e., that the state machines do not communicate. In the figure, actor A has input i_1 and output o_1, and actor B has input i_2 and output o_2. The composition of the two actors is itself an actor C with inputs i_1 and i_2 and outputs o_1 and o_2.[1]

In the simplest scenario, if the two actors are extended state machines with variables, then those variables are also disjoint. We will later consider what happens when the two state machines share variables. Under **synchronous composition**, a reaction of C is a simultaneous reaction of A and B.

Example 5.3: Consider FSMs A and B in Figure 5.3. A has a single pure output a, and B has a single pure output b. The side-by-side composition C has two pure outputs, a and b. If the composition is synchronous, then on the first reaction, a will be *absent* and b will be *present*. On the second reaction, it will be the reverse. On subsequent reactions, a and b will continue to alternate being present.

[1] The composition actor C may rename these input and output ports, but here we assume it uses the same names as the component actors.

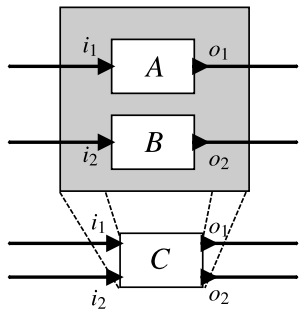


Figure 5.2: Side-by-side composition of two actors.

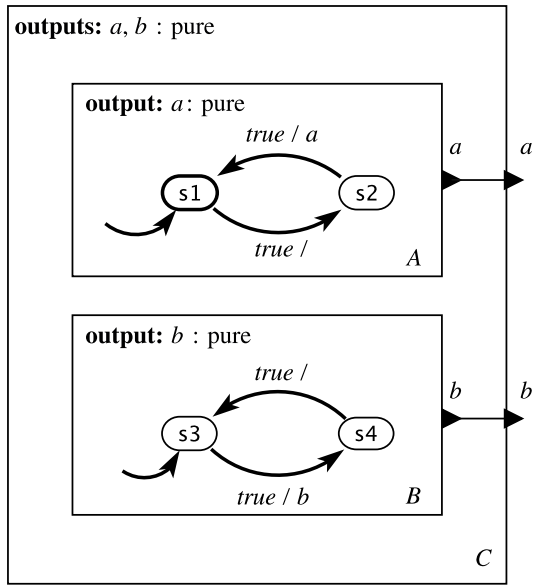


Figure 5.3: Example of side-by-side composition of two actors.

Synchronous side-by-side composition is simple for several reasons. First, recall from Section 3.3.2 that the environment determines when a state machine reacts. In synchronous side-by-side composition, the environment need not be aware that C is a composition of two state machines. Such compositions are **modular** in the sense that the composition itself becomes a component that can be further composed as if it were itself an atomic component.

Moreover, if the two state machines A and B are deterministic, then the synchronous side-by-side composition is also deterministic. We say that a property is **compositional** if a property held by the components is also a property of the composition. For synchronous side-by-side composition, determinism is a compositional property.

In addition, a synchronous side-by-side composition of finite state machines is itself an FSM. A rigorous way to give the semantics of the composition is to define a single state machine for the composition. Suppose that as in Section 3.3.3, state machines A and B

are given by the five tuples,

$$\begin{aligned} A &= (States_A, Inputs_A, Outputs_A, update_A, initialState_A) \\ B &= (States_B, Inputs_B, Outputs_B, update_B, initialState_B) \ . \end{aligned}$$

Then the synchronous side-by-side composition C is given by

$$States_C = States_A \times States_B \tag{5.1}$$

$$Inputs_C = Inputs_A \times Inputs_B \tag{5.2}$$

$$Outputs_C = Outputs_A \times Outputs_B \tag{5.3}$$

$$initialState_C = (initialState_A, initialState_B) \tag{5.4}$$

and the update function is defined by

$$update_C((s_A, s_B), (i_A, i_B)) = ((s'_A, s'_B), (o_A, o_B)),$$

where

$$(s'_A, o_A) = update_A(s_A, i_A),$$

and

$$(s'_B, o_B) = update_B(s_B, i_B),$$

for all $s_A \in States_A$, $s_B \in States_B$, $i_A \in Inputs_A$, and $i_B \in Inputs_B$.

Recall that $Inputs_A$ and $Inputs_B$ are sets of valuations. Each valuation in the set is an assignment of values to ports. What we mean by

$$Inputs_C = Inputs_A \times Inputs_B$$

is that a valuation of the inputs of C must include *both* valuations for the inputs of A and the inputs of B.

As usual, the single FSM C can be given pictorially rather than symbolically, as illustrated in the next example.

Example 5.4: The synchronous side-by-side composition C in Figure 5.3 is given as a single FSM in Figure 5.4. Notice that this machine behaves exactly as described in Example 5.3. The outputs a and b alternate being present. Notice further that $(\mathsf{s1}, \mathsf{s4})$ and $(\mathsf{s2}, \mathsf{s3})$ are not reachable states.

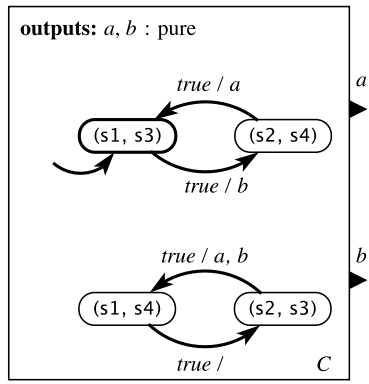


Figure 5.4: Single state machine giving the semantics of synchronous side-by-side composition of the state machines in Figure 5.3.

5.1.2 Side-by-Side Asynchronous Composition

In an **asynchronous composition** of state machines, the component machines react independently. This statement is rather vague, and in fact, it has several different interpretations. Each interpretation gives a semantics to the composition. The key to each semantics is how to define a reaction of the composition C in Figure 5.2. Two possibilities are:

- **Semantics 1.** A reaction of C is a reaction of one of A or B, where the choice is nondeterministic.
- **Semantics 2.** A reaction of C is a reaction of A, B, or both A and B, where the choice is nondeterministic. A variant of this possibility might allow *neither* to react.

Semantics 1 is referred to as an **interleaving semantics**, meaning that A or B never react simultaneously. Their reactions are interleaved in some order.

A significant subtlety is that under these semantics machines A and B may completely miss input events. That is, an input to C destined for machine A may be present in a reaction where the nondeterministic choice results in B reacting rather than A. If this is not desirable, then some control over scheduling (see sidebar on page 118) or synchronous composition becomes a better choice.

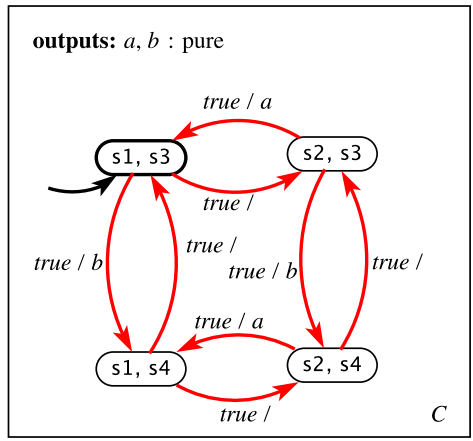


Figure 5.5: State machine giving the semantics of asynchronous side-by-side composition of the state machines in Figure 5.3.

Example 5.5: For the example in Figure 5.3, semantics 1 results in the composition state machine shown in Figure 5.5. This machine is nondeterministic. From state $(\mathsf{s1}, \mathsf{s3})$, when C reacts, it can move to $(\mathsf{s2}, \mathsf{s3})$ and emit no output, or it can move to $(\mathsf{s1}, \mathsf{s4})$ and emit b. Note that if we had chosen semantics 2, then it would also be able to move to $(\mathsf{s2}, \mathsf{s4})$.

For asynchronous composition under semantics 1, the symbolic definition of C has the same definitions of $States_C$, $Inputs_C$, $Outputs_C$, and $initialState_C$ as for synchronous composition, given in (5.1) through (5.4). But the update function differs, becoming

$$update_C((s_A, s_B), (i_A, i_B)) = ((s'_A, s'_B), (o'_A, o'_B)),$$

where either

$$(s'_A, o'_A) = update_A(s_A, i_A) \text{ and } s'_B = s_B \text{ and } o'_B = absent$$

or

$$(s'_B, o'_B) = update_B(s_B, i_B) \text{ and } s'_A = s_A \text{ and } o'_A = absent$$

for all $s_A \in States_A$, $s_B \in States_B$, $i_A \in Inputs_A$, and $i_B \in Inputs_B$. What we mean by $o'_B = absent$ is that all outputs of B are absent. Semantics 2 can be similarly defined (see Exercise 2).

Scheduling Semantics for Asynchronous Composition

In the case of semantics 1 and 2 given in Section 5.1.2, the choice of which component machine reacts is nondeterministic. The model does not express any particular constraints. It is often more useful to introduce some scheduling policies, where the environment is able to influence or control the nondeterministic choice. This leads to two additional possible semantics for asynchronous composition:

- **Semantics 3.** A reaction of C is a reaction of one of A or B, where the environment chooses which of A or B reacts.
- **Semantics 4.** A reaction of C is a reaction of A, B, or both A and B, where the choice is made by the environment.

Like semantics 1, semantics 3 is an interleaving semantics.

In one sense, semantics 1 and 2 are more compositional than semantics 3 and 4. To implement semantics 3 and 4, a composition has to provide some mechanism for the environment to choose which component machine should react (for scheduling the component machines). This means that the hierarchy suggested in Figure 5.2 does not quite work. Actor C has to expose more of its internal structure than just the ports and the ability to react.

In another sense, semantics 1 and 2 are less compositional than semantics 3 and 4 because determinism is not preserved by composition. A composition of deterministic state machines is not a deterministic state machine.

Notice further that semantics 1 is an abstraction of semantics 3 in the sense that every behavior under semantics 3 is also a behavior under semantics 1. This notion of abstraction is studied in detail in Chapter 14.

The subtle differences between these choices make asynchronous composition rather treacherous. Considerable care is required to ensure that it is clear which semantics is used.

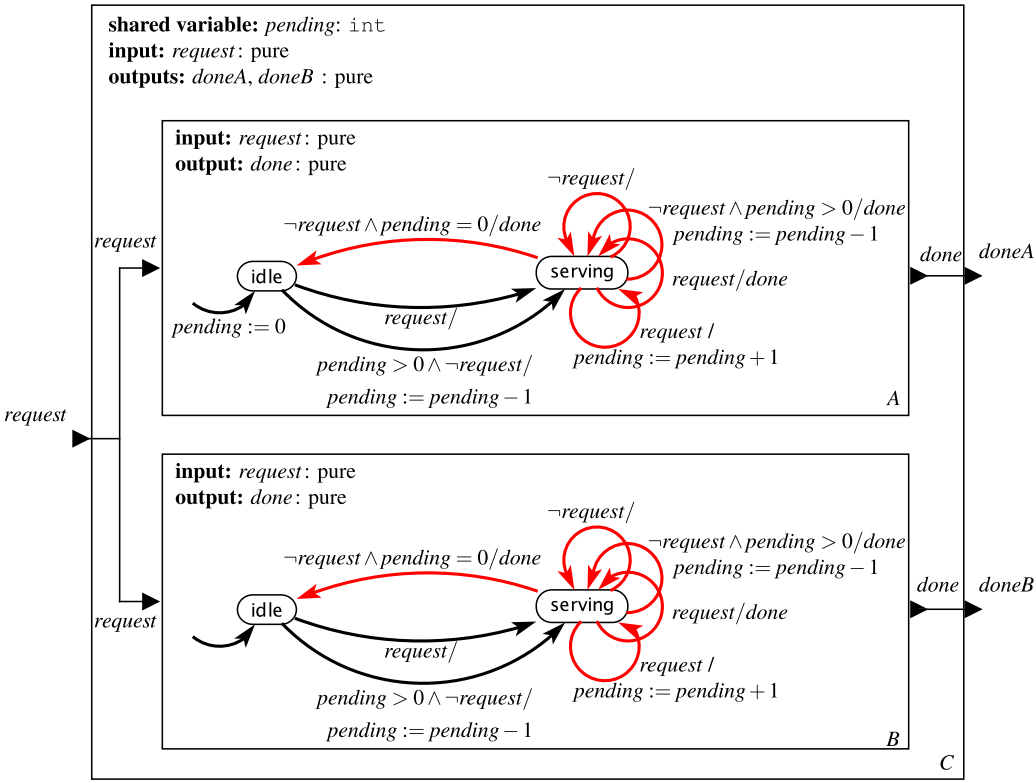


Figure 5.6: Model of two servers with a shared task queue, assuming asynchronous composition under semantics 1.

5.1.3 Shared Variables

An extended state machine has local variables that can be read and written as part of taking a transition. Sometimes it is useful when composing state machines to allow these variables to be shared among a group of machines. In particular, such shared variables can be useful for modeling interrupts, studied in Chapter 10, and threads, studied in Chapter 11. However, considerable care is required to ensure that the semantics of the model conforms with that of the program containing interrupts or threads. Many complications arise, including the memory consistency model and the notion of atomic operations.

Example 5.6: Consider two servers that can receive requests from a network. Each request requires an unknown amount of time to service, so the servers share a queue of requests. If one server is busy, the other server can respond to a request, even if the request arrives at the network interface of the first server.

This scenario fits a pattern similar to that in Figure 5.2, where A and B are the servers. We can model the servers as state machines as shown in Figure 5.6. In this model, a shared variable *pending* counts the number of pending job requests. When a request arrives at the composite machine C, one of the two servers is nondeterministically chosen to react, assuming asynchronous composition under semantics 1. If that server is idle, then it proceeds to serve the request. If the server is serving another request, then one of two things can happen: it can coincidentally finish serving the request it is currently serving, issuing the output *done*, and proceed to serve the new one, or it can increment the count of pending requests and continue to serve the current request. The choice between these is nondeterministic, to model the fact that the time it takes to service a request is unknown.

If C reacts when there is no request, then again either server A or B will be selected nondeterministically to react. If the server that reacts is idle and there are one or more pending requests, then the server transitions to serving and decrements the variable *pending*. If the server that reacts is not idle, then one of three things can happen. It may continue serving the current request, in which case it simply transitions on the self transition back to serving. Or it may finish serving the request, in which case it will transition to idle if there are no pending requests, or transition back to serving and decrement *pending* if there are pending requests.

The model in the previous example exhibits many subtleties of concurrent systems. First, because of the interleaving semantics, accesses to the shared variable are atomic operations, something that is quite challenging to guarantee in practice, as discussed in Chapters 10 and 11. Second, the choice of semantics 1 is reasonable in this case because the input goes to both of the component machines, so regardless of which component machine reacts, no input event will be missed. However, this semantics would not work if the two machines had independent inputs, because then requests could be missed. Semantics 2 can help prevent that, but what strategy should be used by the environment to determine

which machine reacts? What if the two independent inputs both have requests present at the same reaction of C? If we choose semantics 4 in the sidebar on page 118 to allow both machines to react simultaneously, then what is the meaning when both machines update the shared variable? The updates are no longer atomic, as they are with an interleaving semantics.

Note further that choosing asynchronous composition under semantics 1 allows behaviors that do not make good use of idle machines. In particular, suppose that machine A is serving, machine B is idle, and a *request* arrives. If the nondeterministic choice results in machine A reacting, then it will simply increment *pending*. Not until the nondeterministic choice results in B reacting will the idle machine be put to use. In fact, semantics 1 allows behaviors that never use one of the machines.

Shared variables may be used in synchronous compositions as well, but sophisticated subtleties again emerge. In particular, what should happen if in the same reaction one machine reads a shared variable to evaluate a guard and another machine writes to the shared variable? Do we require the write before the read? What if the transition doing the write to the shared variable also reads the same variable in its guard expression? One possibility is to choose a **synchronous interleaving semantics**, where the component machines react in arbitrary order, chosen nondeterministically. This strategy has the disadvantage that a composition of two deterministic machines may be nondeterministic. An alternative version of the synchronous interleaving semantics has the component machines react in a fixed order determined by the environment or by some additional mechanism such as priority.

The difficulties of shared variables, particularly with asynchronous composition, reflect the inherent complexity of concurrency models with shared variables. Clean solutions require a more sophisticated semantics, to be discussed in Chapter 6. In that chapter, we will explain the synchronous-reactive model of computation, which gives a synchronous composition semantics that is reasonably compositional.

So far, we have considered composition of machines that do not directly communicate. We next consider what happens when the outputs of one machine are the inputs of another.

5.1.4 Cascade Composition

Consider two state machines A and B that are composed as shown in Figure 5.7. The output of machine A feeds the input of B. This style of composition is called cascade composition or **serial composition**.

In the figure, output port o_1 from A feeds events to input port i_2 of B. Assume the data type of o_1 is V_1 (meaning that o_1 can take values from V_1 or be *absent*), and the data type of i_2 is V_2. Then a requirement for this composition to be valid is that

$$V_1 \subseteq V_2 \ .$$

This asserts that any output produced by A on port o_1 is an acceptable input to B on port i_2. The composition **type check**s.

For cascade composition, if we wish the composition to be asynchronous, then we need to introduce some machinery for buffering the data that is sent from A to B. We defer discussion of such asynchronous composition to Chapter 6, where dataflow and process network models of computation will provide such asynchronous composition. In this chapter, we will only consider synchronous composition for cascade systems.

In synchronous composition of the cascade structure of Figure 5.7, a reaction of C consists of a reaction of both A and B, where A reacts first, produces its output (if any), and then B reacts. Logically, we view this as occurring in zero time, so the two reactions are in a sense **simultaneous and instantaneous**. But they are causally related in that the outputs of A can affect the behavior of B.

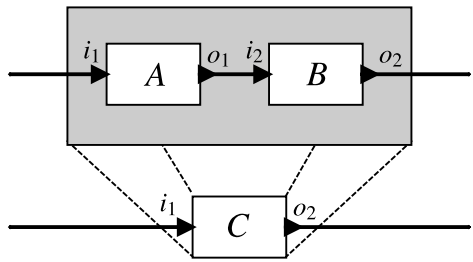


Figure 5.7: Cascade composition of two actors.

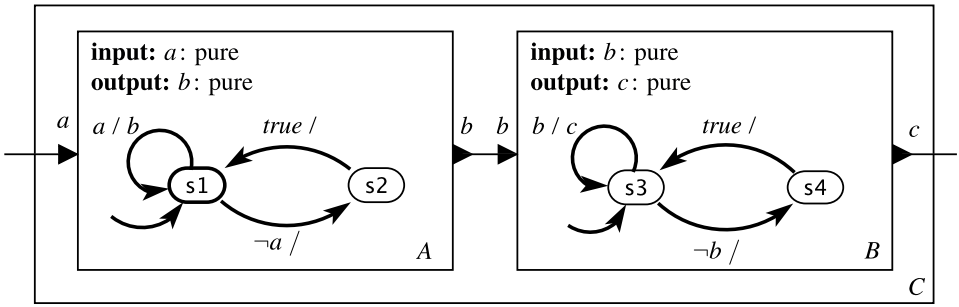


Figure 5.8: Example of a cascade composition of two FSMs.

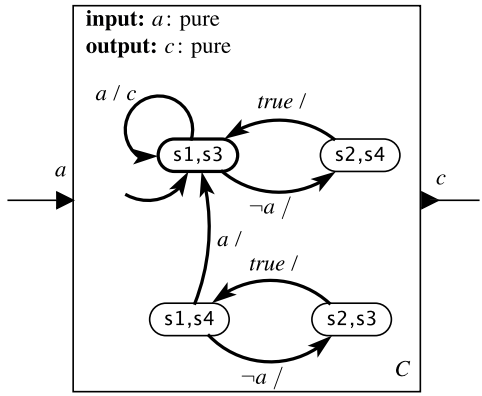


Figure 5.9: Semantics of the cascade composition of Figure 5.8, assuming synchronous composition.

Example 5.7: Consider the cascade composition of the two FSMs in Figure 5.8. Assuming synchronous semantics, the meaning of a reaction of C is given in Figure 5.9. That figure makes it clear that the reactions of the two machines are simultaneous and instantaneous. When moving from the initial state (s1, s3) to (s2, s4) (which occurs when the input a is absent), the composition machine C

does not pass through (s2, s3)! In fact, (s2, s3) is not a reachable state! In this way, a *single* reaction of C encompasses a reaction of both A *and* B.

To construct the composition machine as in Figure 5.9, first form the state space as the cross product of the state spaces of the component machines, and then determine which transitions are taken under what conditions. It is important to remember that the transitions are simultaneous, even when one logically causes the other.

Example 5.8: Recall the traffic light model of Figure 3.10. Suppose that we wish to compose this with a model of a pedestrian crossing light, like that shown in Figure 5.10. The output *sigR* of the traffic light can provide the input *sigR* of the pedestrian light. Under synchronous cascade composition, the meaning of the composite is given in Figure 5.11. Note that unsafe states, such as (green, green), which is the state when both cars and pedestrians have a green light, are not reachable states, and hence are not shown.

In its simplest form, cascade composition implies an ordering of the reactions of the

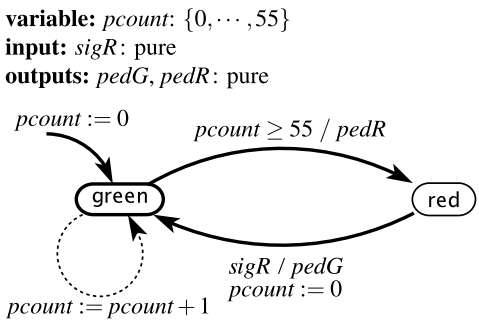


Figure 5.10: A model of a pedestrian crossing light, to be composed in a synchronous cascade composition with the traffic light model of Figure 3.10.

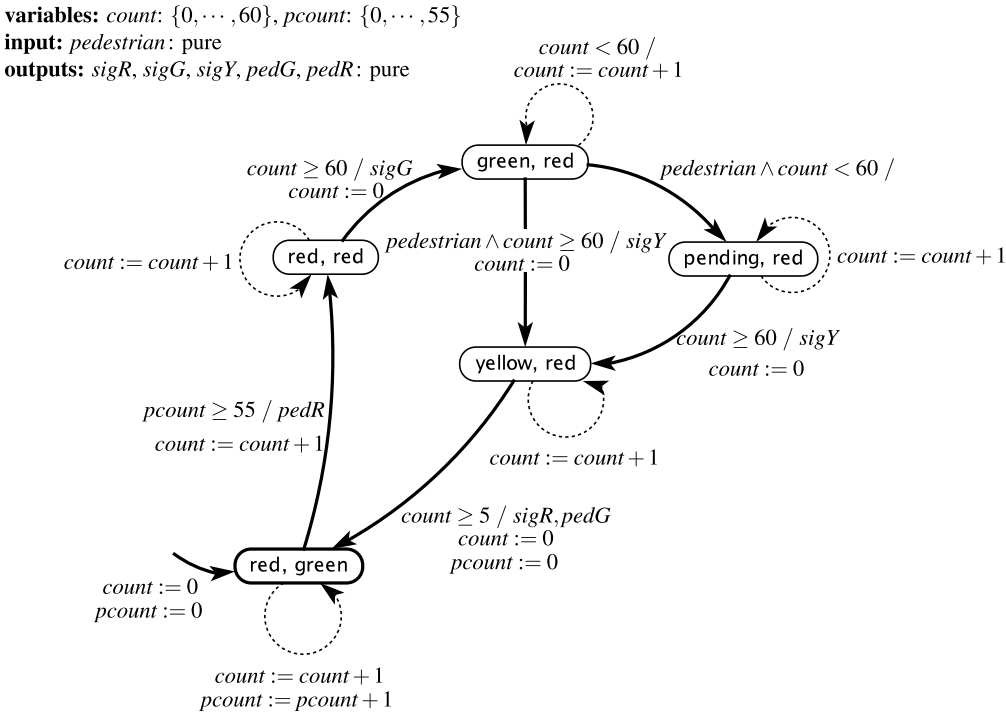


Figure 5.11: Semantics of a synchronous cascade composition of the traffic light model of Figure 3.10 with the pedestrian light model of Figure 5.10.

components. Since this ordering is well defined, we do not have as much difficulty with shared variables as we did with side-by-side composition. However, we will see that in more general compositions, the ordering is not so simple.

5.1.5 General Composition

Side-by-side and cascade composition provide the basic building blocks for building more complex compositions of machines. Consider for example the composition in Figure 5.12. A_1 and A_3 are a side-by-side composition that together define a machine B. B and A_2 are a cascade composition, with B feeding events to A_2. However, B and A_2 are also a cascade composition in the opposite order, with A_2 feeding events to B. Cycles like

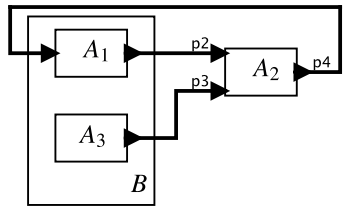


Figure 5.12: Arbitrary interconnections of state machines are combinations of side-by-side and cascade compositions, possibly creating cycles, as in this example.

this are called feedback, and they introduce a conundrum; which machine should react first, B or A_2? This conundrum will be resolved in the next chapter when we explain the synchronous-reactive model of computation.

5.2 Hierarchical State Machines

In this section, we consider **hierarchical FSMs**, which date back to Statecharts (Harel, 1987). There are many variants of Statecharts, often with subtle semantic differences between them (von der Beeck, 1994). Here, we will focus on some of the simpler aspects only, and we will pick a particular semantic variant.

The key idea in hierarchical state machines is state refinement. In Figure 5.13, state B has a refinement that is another FSM with two states, C and D. What it means for the machine to be in state B is that it is in one of states C or D.

The meaning of the hierarchy in Figure 5.13 can be understood by comparing it to the equivalent flattened FSM in Figure 5.14. The machine starts in state A. When guard g_2 evaluates to true, the machine transitions to state B, which means a transition to state C, the initial state of the refinement. Upon taking this transition to C, the machine performs action a_2, which may produce an output event or set a variable (if this is an extended state machine).

There are then two ways to exit C. Either guard g_1 evaluates to true, in which case the machine exits B and returns to A, or guard g_4 evaluates to true and the machine transitions to D. A subtle question is what happens if both guards g_1 and g_4 evaluate to true. Different

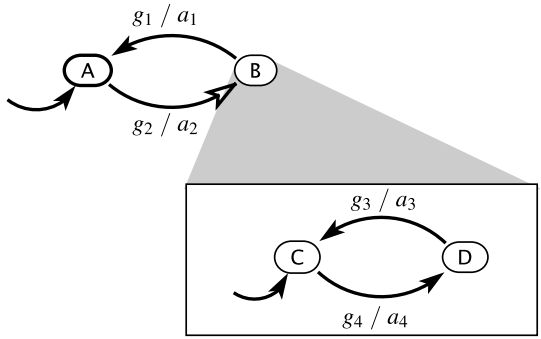


Figure 5.13: In a hierarchical FSM, a state may have a refinement that is another state machine.

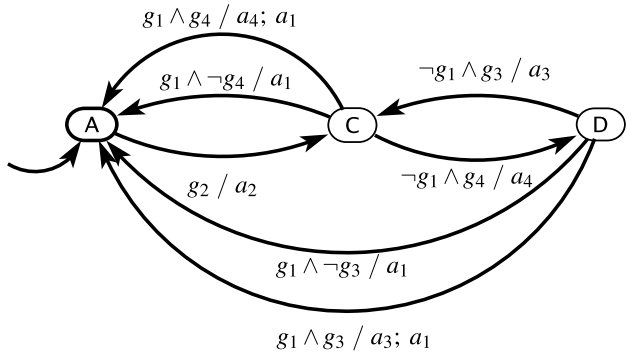


Figure 5.14: Semantics of the hierarchical FSM in Figure 5.13.

variants of Statecharts may make different choices at this point. It seems reasonable that the machine should end up in state A, but which of the actions should be performed, a_4, a_1, or both? Such subtle questions help account for the proliferation of different variants of Statecharts.

We choose a particular semantics that has attractive modularity properties (Lee and Tripakis, 2010). In this semantics, a reaction of a hierarchical FSM is defined in a depth-first fashion. The deepest refinement of the current state reacts first, then its container state machine, then its container, etc. In Figure 5.13, this means that if the machine is in state

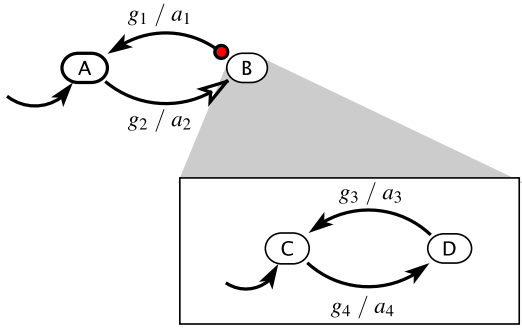


Figure 5.15: Variant of Figure 5.13 that uses a preemptive transition.

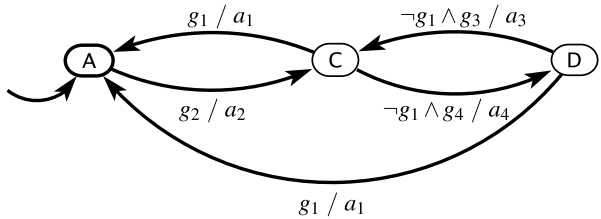


Figure 5.16: Semantics of Figure 5.15 with a preemptive transition.

B (which means that it is in either C or D), then the refinement machine reacts first. If it is C, and guard g_4 is true, the transition is taken to D and action a_4 is performed. But then, as part of the same reaction, the top-level FSM reacts. If guard g_1 is also true, then the machine transitions to state A. It is important that logically these two transitions are simultaneous and instantaneous, so the machine does not actually go to state D. Nonetheless, action a_4 is performed, and so is action a_1. This combination corresponds to the topmost transition of Figure 5.14.

Another subtlety is that if two (non-absent) actions are performed in the same reaction, they may conflict. For example, two actions may write different values to the same output port. Or they may set the same variable to different values. Our choice is that the actions are performed in sequence, as suggested by the semicolon in the action a_4; a_1. As in an imperative language like C, the semicolon denotes a sequence. If the two actions conflict, the later one dominates.

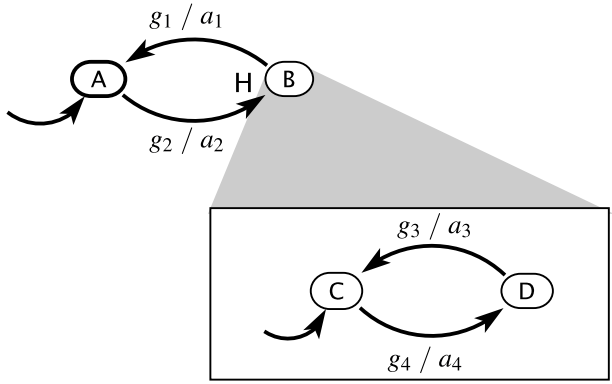


Figure 5.17: Variant of the hierarchical state machine of Figure 5.13 that has a history transition.

Such subtleties can be avoided by using a **preemptive transition**, shown in Figure 5.15, which has the semantics shown in Figure 5.16. The guards of a preemptive transition are evaluated *before* the refinement reacts, and if any guard evaluates to true, the refinement does not react. As a consequence, if the machine is in state B and g_1 is true, then neither action a_3 nor a_4 is performed. A preemptive transition is shown with a (red) circle at the originating end of the transition.

Notice in Figures 5.13 and 5.14 that whenever the machine enters B, it always enters C, never D, even if it was previously in D when leaving B. The transition from A to B is called a **reset transition** because the destination refinement is reset to its initial state, regardless of where it had previously been. A reset transition is indicated in our notation with a hollow arrowhead at the destination end of a transition.

In Figure 5.17, the transition from A to B is a **history transition**, an alternative to a reset transition. In our notation, a solid arrowhead denotes a history transition. It may also be marked with an "H" for emphasis. When a history transition is taken, the destination refinement resumes in whatever state it was last in (or its initial state on the first entry).

The semantics of the history transition is shown in Figure 5.18. The initial state is labeled (A, C) to indicate that the machine is in state A, and if and when it next enters B it will go

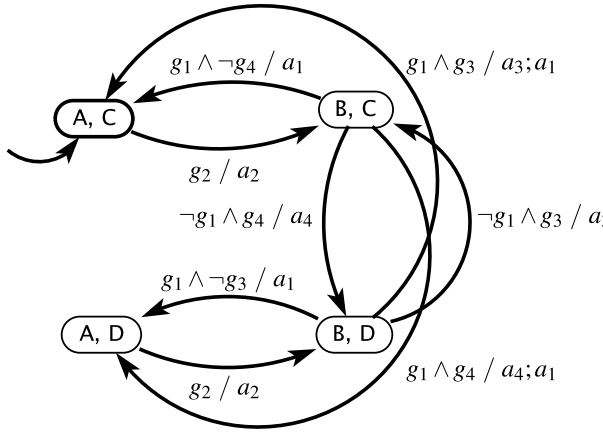


Figure 5.18: Semantics of the hierarchical state machine of Figure 5.17 that has a history transition.

to C. The first time it goes to B, it will be in the state labeled (B, C) to indicate that it is in state B and, more specifically, C. If it then transitions to (B, D), and then back to A, it will end up in the state labeled (A, D), which means it is in state A, but if and when it next enters B it will go to D. That is, it remembers its history, specifically where it was when it left B.

As with concurrent composition, hierarchical state machines admit many possible meanings. The differences can be subtle. Considerable care is required to ensure that models are clear and that their semantics match what is being modeled.

5.3 Summary

Any well-engineered system is a composition of simpler components. In this chapter, we have considered two forms of composition of state machines, concurrent composition and hierarchical composition.

For concurrent composition, we introduced both synchronous and asynchronous composition, but did not complete the story. We have deferred dealing with feedback to the

next chapter, because for synchronous composition, significant subtleties arise. For asynchronous composition, communication via ports requires additional mechanisms that are not (yet) part of our model of state machines. Even without communication via ports, significant subtleties arise because there are several possible semantics for asynchronous composition, and each has strengths and weaknesses. One choice of semantics may be suitable for one application and not for another. These subtleties motivate the topic of the next chapter, which provides more structure to concurrent composition and resolves most of these questions (in a variety of ways).

For hierarchical composition, we focus on a style originally introduced by Harel (1987) known as Statecharts. We specifically focus on the ability for states in an FSM to have refinements that are themselves state machines. The reactions of the refinement FSMs are composed with those of the machine that contains the refinements. As usual, there are many possible semantics.

Exercises

1. Consider the extended state machine model of Figure 3.8, the garage counter. Suppose that the garage has two distinct entrance and exit points. Construct a side-by-side concurrent composition of two counters that share a variable c that keeps track of the number of cars in the garage. Specify whether you are using synchronous or asynchronous composition, and define exactly the semantics of your composition by giving a single machine modeling the composition. If you choose synchronous semantics, explain what happens if the two machines simultaneously modify the shared variable. If you choose asynchronous composition, explain precisely which variant of asynchronous semantics you have chosen and why. Is your composition machine deterministic?

2. For semantics 2 in Section 5.1.2, give the five tuple for a single machine representing the composition C,

$$(States_C, Inputs_C, Outputs_C, update_C, initialState_C)$$

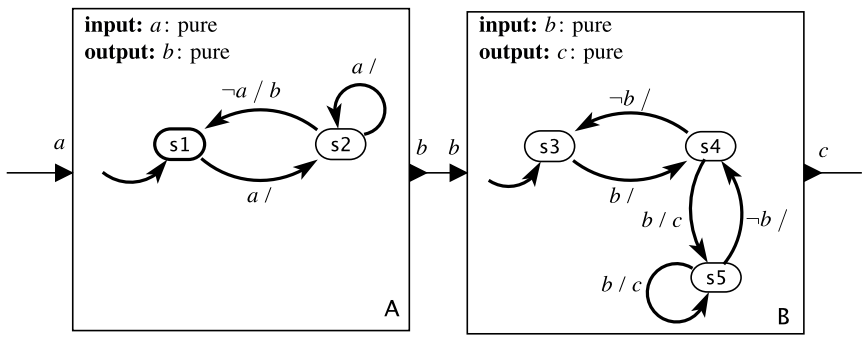
for the side-by-side asynchronous composition of two state machines A and B. Your answer should be in terms of the five-tuple definitions for A and B,

$$(States_A, Inputs_A, Outputs_A, update_A, initialState_A)$$

and

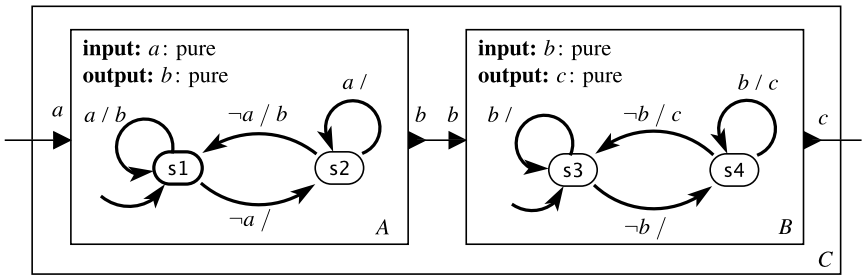
$$(States_B, Inputs_B, Outputs_B, update_B, initialState_B)$$

3. Consider the following synchronous composition of two state machines A and B:



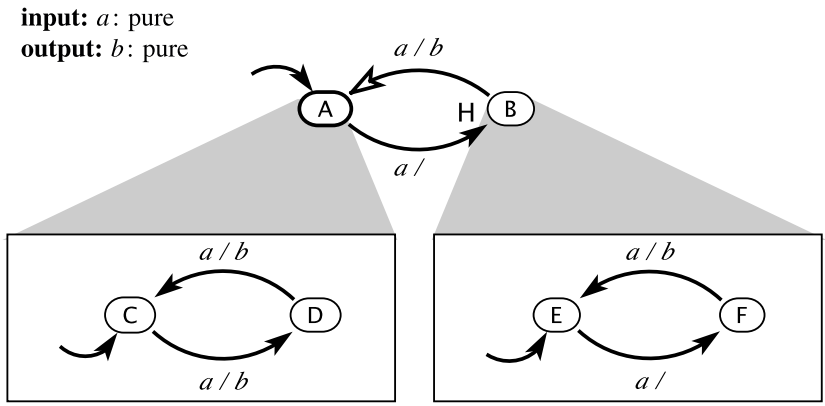
Construct a single state machine C representing the composition. Which states of the composition are unreachable?

4. Consider the following synchronous composition of two state machines A and B:



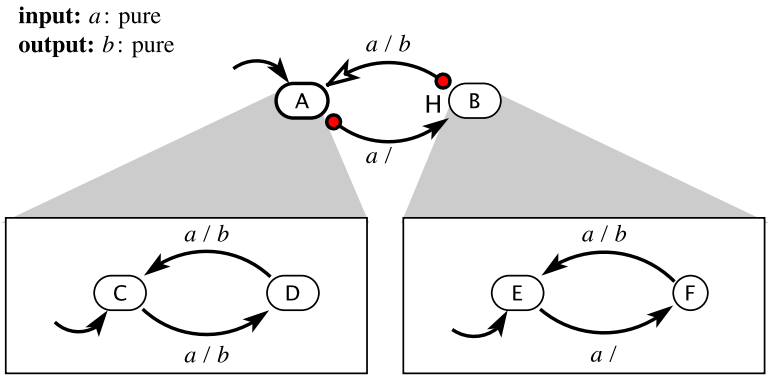
Construct a single state machine C representing the composition. Which states of the composition are unreachable?

5. Consider the following hierarchical state machine:

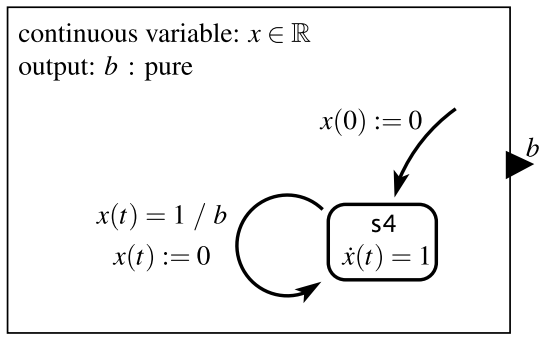


Construct an equivalent flat FSM giving the semantics of the hierarchy. Describe in words the input/output behavior of this machine. Is there a simpler machine that exhibits the same behavior? (Note that equivalence relations between state machines are considered in Chapter 14, but here, you can use intuition and just consider what the state machine does when it reacts.)

6. How many reachable states does the following state machine have?



7. Suppose that the machine of Exercise 8 of Chapter 4 is composed in a synchronous side-by-side composition with the following machine:



Find a tight lower bound on the time between events a and b. That is, find a lower bound on the time gap during which there are no events in signals a or b. Give an argument that your lower bound is tight.

6 Concurrent Models of Computation

6.1 Structure of Models . . . 137
Sidebar: Actor Networks as a System of Equations . . . 139
Sidebar: Fixed-Point Semantics . . . 140
6.2 Synchronous-Reactive Models . . . 141
6.2.1 Feedback Models . . . 141
6.2.2 Well-Formed and Ill-Formed Models . . . 143
6.2.3 Constructing a Fixed Point . . . 145
6.3 Dataflow Models of Computation . . . 147
Sidebar: Synchronous-Reactive Languages . . . 148
6.3.1 Dataflow Principles . . . 149
6.3.2 Synchronous Dataflow . . . 152
6.3.3 Dynamic Dataflow . . . 157
6.3.4 Structured Dataflow . . . 158
6.3.5 Process Networks . . . 159
6.4 Timed Models of Computation . . . 162
6.4.1 Time-Triggered Models . . . 162
6.4.2 Discrete Event Systems . . . 163
6.4.3 Continuous-Time Systems . . . 164
6.5 Summary . . . 167
Sidebar: Petri Nets . . . 168
Sidebar: Models of Time . . . 169
Sidebar: Probing Further: Discrete Event Semantics . . . 170
Exercises . . . 172

In sound engineering practice, systems are built by composing components. In order for the composition to be well understood, we need first for the individual components to be well understood, and then for the meaning of the interaction between components to be well understood. The previous chapter dealt with composition of finite state machines. With such composition, the components are well defined (they are FSMs), but there are many possible interpretations for the interaction between components. The meaning of a composition is referred to as its semantics.

This chapter focuses on the semantics of **concurrent** composition. The word "concurrent" literally means "running together." A system is said to be concurrent if different parts of the system (components) conceptually operate at the same time. There is no particular order to their operations. The semantics of such concurrent operation can be quite subtle, however.

The components we consider in this chapter are actors, which react to stimuli at input ports and produce stimuli on output ports. In this chapter, we will be only minimally concerned with how the actors themselves are defined. They may be FSMs, hardware, or programs specified in an imperative programming language. We will need to impose some constraints on what these actors can do, but we need not constrain how they are specified.

The semantics of a concurrent composition of actors is governed by three sets of rules that we collectively call a **model of computation** (**MoC**). The first set of rules specifies what constitutes a component. The second set specifies the concurrency mechanisms. The third specifies the communication mechanisms.

In this chapter, a component will be an actor with ports and a set of **execution actions**. An execution action defines how the actor reacts to inputs to produce outputs and change state. The ports will be interconnected to provide for communication between actors, and the execution actions will be invoked by the environment of the actor to cause the actor to perform its function. For example, for FSMs, one action is provided that causes a reaction. The focus of this chapter is on introducing a few of the possible concurrency and communication mechanisms that can govern the interactions between such actors.

We begin by laying out the common structure of models that applies to all MoCs studied in this chapter. We then proceed to describe a suite of MoCs.

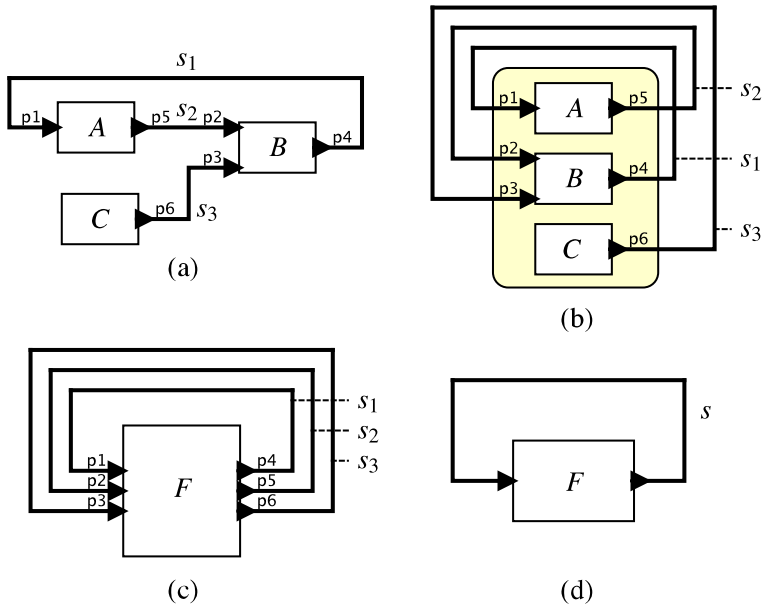


Figure 6.1: Any interconnection of actors can be modeled as a single (side-by-side composite) actor with feedback.

6.1 Structure of Models

In this chapter, we assume that models consist of fixed interconnections of actors like that shown in Figure 6.1(a). The interconnections between actors specify communication paths. The communication itself takes the form of a **signal**, which consists of one or more **communication events**. For the discrete signals of Section 3.1, for example, a signal s has the form of a function

$$s : \mathbb{R} \rightarrow V_s \cup \{absent\},$$

where V_s is a set of values called the type of the signal s. A communication event in this case is a non-absent value of s.

Example 6.1: Consider a pure signal s that is a discrete signal given by

$$s(t) = \begin{cases} \textit{present} & \text{if } t \text{ is a multiple of } P \\ \textit{absent} & \text{otherwise} \end{cases}$$

for all $t \in \mathbb{R}$ and some $P \in \mathbb{R}$. Such a signal is called a **clock signal** with period P. Communication events occur every P time units.

In Chapter 2, a continuous-time signal has the form of a function

$$s : \mathbb{R} \to V_s,$$

in which case every one of the (uncountably) infinite set of values $s(t)$, for all $t \in \mathbb{R}$, is a communication event. In this chapter, we will also encounter signals of the form

$$s : \mathbb{N} \to V_s,$$

where there is no time line. The signal is simply a sequence of values.

A communication event has a type, and we require that a connection between actors type check. That is, if an output port y with type V_y is connected to an input port x with type V_x, then

$$V_y \subseteq V_x.$$

As suggested in Figure 6.1(b-d), any actor network can be reduced to a rather simple form. If we rearrange the actors as shown in Figure 6.1(b), then the actors form a side-by-side composition indicated by the box with rounded corners. This box is itself an actor F as shown in Figure 6.1(c) whose input is a three-tuple (s_1, s_2, s_3) of signals and whose output is *the same* three-tuple of signals. If we let $s = (s_1, s_2, s_3)$, then the actor can be depicted as in Figure 6.1(d), which hides all the complexity of the model.

Notice that Figure 6.1(d) is a feedback system. By following the procedure that we used to build it, every interconnection of actors can be structured as a similar feedback system (see Exercise 1).

Actor Networks as a System of Equations

In a model, if the actors are determinate, then each actor is a function that maps input signals to output signals. For example, in Figure 6.1(a), actor A may be a function relating signals s_1 and s_2 as follows,

$$s_2 = A(s_1).$$

Similarly, actor B relates three signals by

$$s_1 = B(s_2, s_3).$$

Actor C is a bit more subtle, since it has no input ports. How can it be a function? What is the domain of the function? If the actor is determinate, then its output signal s_3 is a constant signal. The function C needs to be a constant function, one that yields the same output for every input. A simple way to ensure this is to define C so that its domain is a singleton set (a set with only one element). Let $\{\emptyset\}$ be the singleton set, so C can only be applied to $\emptyset$. The function C is then given by

$$C(\emptyset) = s_3.$$

Hence, Figure 6.1(a) gives a system of equations

$$\begin{aligned} s_1 &= B(s_2, s_3) \\ s_2 &= A(s_1) \\ s_3 &= C(\emptyset). \end{aligned}$$

The semantics of such a model, therefore, is a solution to such a system of equations. This can be represented compactly using the function F in Figure 6.1(d), which is

$$F(s_1, s_2, s_3) = (B(s_2, s_3), A(s_1), C(\emptyset)).$$

All actors in Figure 6.1(a) have output ports; if we had an actor with no output port, then we could similarly define it as a function whose codomain is $\{\emptyset\}$. The output of such function is $\emptyset$ for all inputs.

Fixed-Point Semantics

In a model, if the actors are determinate, then each actor is a function that maps input signals to output signals. The semantics of such a model is a system of equations (see sidebar on page 139) and the reduced form of Figure 6.1(d) becomes

$$s = F(s), \tag{6.1}$$

where $s = (s_1, s_2, s_3)$. Of course, this equation only *looks* simple. Its complexity lies in the definition of the function F and the structure of the domain and range of F.

Given any function $F: X \to X$ for any set X, if there is an $x \in X$ such that $F(x) = x$, then x is called a **fixed point**. Equation (6.1) therefore asserts that the semantics of a determinate actor network is a fixed point. Whether a fixed point exists, whether the fixed point is unique, and how to find the fixed point, all become interesting questions that are central to the model of computation.

In the SR model of computation, the execution of all actors is simultaneous and instantaneous and occurs at ticks of the global clock. If the actor is determinate, then each such execution implements a function called a **firing function**. For example, in the n-th tick of the global clock, actor A in Figure 6.1 implements a function of the form

$$a_n: V_1 \cup \{absent\} \to V_2 \cup \{absent\}$$

where V_i is the type of signal s_i. Hence, if $s_i(n)$ is the value of s_i at the n-th tick, then

$$s_2(n) = a_n(s_1(n)).$$

Given such a firing function f_n for each actor F we can, just as in Figure 6.1(d) define the execution at a single tick by a fixed point,

$$s(n) = f_n(s(n)),$$

where $s(n) = (s_1(n), s_2(n), s_3(n))$ and f_n is a function where

$$f_n(s_1(n), s_2(n), s_3(n)) = (b_n(s_2(n), s_3(n)), a_n(s_1(n)), c_n(\emptyset)).$$

Thus, for SR, the semantics at each tick of the global clock is a fixed point of the function f_n, just as its execution over all ticks is a fixed point of the function F.

6.2 Synchronous-Reactive Models

In Chapter 5 we studied synchronous composition of state machines, but we avoided the nuances of feedback compositions. For a model described as the feedback system of Figure 6.1(d), the conundrum discussed in Section 5.1.5 takes a particularly simple form. If F in Figure 6.1(d) is realized by a state machine, then in order for it to react, we need to know its inputs at the time of the reaction. But its inputs are the same as its outputs, so in order for F to react, we need to know its outputs. But we cannot know its outputs until after it reacts.

As shown in Section 6.1 above and Exercise 1, all actor networks can be viewed as feedback systems, so we really do have to resolve the conundrum. We do that now by giving a model of computation known as the **synchronous-reactive** (**SR**) MoC.

An SR model is a discrete system where signals are absent at all times except (possibly) at **ticks** of a **global clock**. Conceptually, execution of a model is a sequence of global reactions that occur at discrete times, and at each such reaction, the reaction of all actors is simultaneous and instantaneous.

6.2.1 Feedback Models

We focus first on feedback models of the form of Figure 6.1(d), where F in the figure is realized as a state machine. At the n-th tick of the global clock, we have to find the value of the signal s so that it is both a valid input and a valid output of the state machine, given its current state. Let $s(n)$ denote the value of the signal s at the n-th reaction. The goal is to determine, at each tick of the global clock, the value of $s(n)$.

Example 6.2: Consider first a simpler example shown in Figure 6.2. (This is simpler than Figure 6.1(d) because the signal s is a single pure signal rather than an aggregation of three signals.) If A is in state `s1` when that reaction occurs, then the only possible value for $s(n)$ is $s(n) = \textit{absent}$ because a reaction must take one of the transitions out of `s1`, and both of these transitions emit absent. Moreover, once we know that $s(n) = \textit{absent}$, we know that the input port x has value *absent*, so we can determine that A will transition to state `s2`.

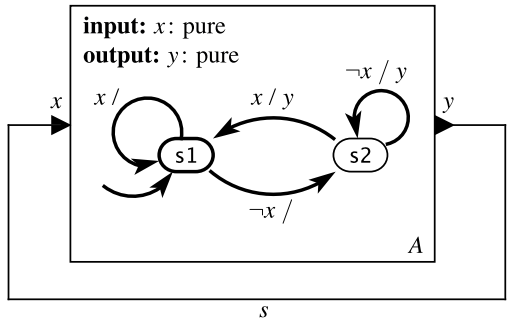


Figure 6.2: A simple well-formed feedback model.

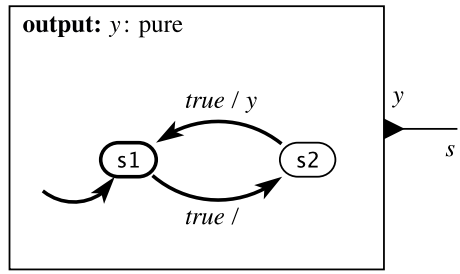


Figure 6.3: The semantics of the model in Figure 6.2.

If A is in state s2 when the reaction occurs, then the only possible value for $s(n)$ is $s(n) = \textit{present}$, and the machine will transition to state s1. Therefore, s alternates between *absent* and *present*. The semantics of machine A in the feedback model is therefore given by Figure 6.3.

In the previous example, it is important to note that the input x and output y have the *same value* in every reaction. This is what is meant by the feedback connection. Any connection from an output port to an input port means that the value at the input port is the same as the value at the output port at all times.

Given a deterministic state machine in a feedback model like that of Figure 6.2, in each state i we can define a function a_i that maps input values to output values,

$$a_i : \{present, absent\} \to \{present, absent\},$$

where the function depends on the state the machine is in. This function is defined by the update function.

Example 6.3: For the example in Figure 6.2, if the machine is in state s1, then $a_{\mathsf{s1}}(x) = absent$ for all $x \in \{present, absent\}$.

The function a_i is called the firing function for state i (see box on page 140). Given a firing function, to find the value $s(n)$ at the n-th reaction, we simply need to find a value $s(n)$ such that

$$s(n) = a_i(s(n)).$$

Such a value $s(n)$ is called a **fixed point** of the function a_i. It is easy to see how to generalize this so that the signal s can have any type. Signal s can even be an aggregation of signals, as in Figure 6.1(d) (see box on page 140).

6.2.2 Well-Formed and Ill-Formed Models

There are two potential problems that may occur when seeking a fixed point. First, there may be no fixed point. Second, there may be more than one fixed point. If either case occurs in a reachable state, we call the system **ill formed**. Otherwise, it is **well formed**.

Example 6.4: Consider machine B shown in Figure 6.4. In state s1, we get the unique fixed point $s(n) = absent$. In state s2, however, there is no fixed point. If we attempt to choose $s(n) = present$, then the machine will transition to s1 and its output will be *absent*. But the output has to be the same as the input, and the input is *present*, so we get a contradiction. A similar contradiction occurs if we attempt to choose $s(n) = absent$.

Since state s2 is reachable, this feedback model is ill formed.

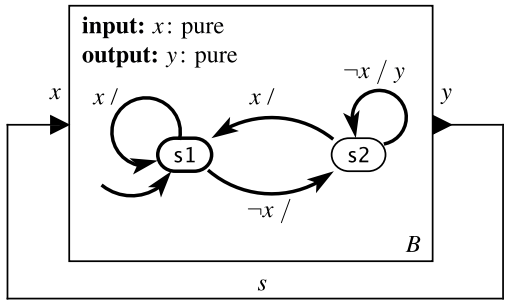


Figure 6.4: An ill-formed feedback model that has no fixed point in state s2.

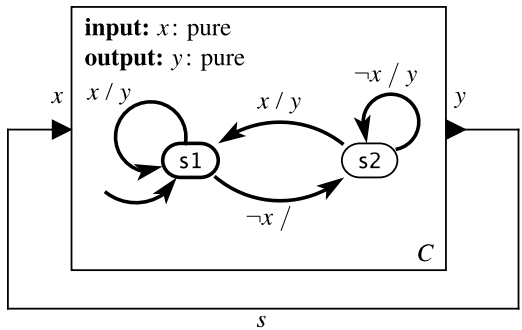


Figure 6.5: An ill-formed feedback model that has more than one fixed point in state s1.

Example 6.5: Consider machine C shown in Figure 6.5. In state s1, both $s(n) =$ *absent* and $s(n) =$ *present* are fixed points. Either choice is valid. Since state s1 is reachable, this feedback model is ill formed.

If in a reachable state there is more than one fixed point, we declare the machine to be ill formed. An alternative semantics would not reject such a model, but rather would declare it to be nondeterministic. This would be a valid semantics, but it would have the

disadvantage that a composition of deterministic state machines is not assured of being deterministic. In fact, C in Figure 6.5 is deterministic, and under this alternative semantics, the feedback composition in the figure would not be deterministic. Determinism would not be a compositional property. Hence, we prefer to reject such models.

6.2.3 Constructing a Fixed Point

If the type V_s of the signal s or the signals it is an aggregate of is finite, then one way to find a fixed point is by **exhaustive search**, which means to try all values. If exactly one fixed point is found, then the model is well formed. However, exhaustive search is expensive (and impossible if the types are not finite). We can develop instead a systematic procedure that for most, but not all, well-formed models will find a fixed point. The procedure is as follows. For each reachable state i,

1. Start with $s(n)$ *unknown*.
2. Determine as much as you can about $f_i(s(n))$, where f_i is the firing function in state i. Note that in this step, you should use only the firing function, which is given by the state machine; you should not use knowledge of how the state machine is connected on the outside.
3. Repeat step 2 until all values in $s(n)$ become known (whether they are present and what their values are), or until no more progress can be made.
4. If unknown values remain, then reject the model.

This procedure may reject models that have a unique fixed point, as illustrated by the following example.

Example 6.6: Consider machine D shown in Figure 6.6. In state `s1`, if the input is unknown, we cannot immediately tell what the output will be. We have to try all the possible values for the input to determine that in fact $s(n) = \textit{absent}$ for all n.

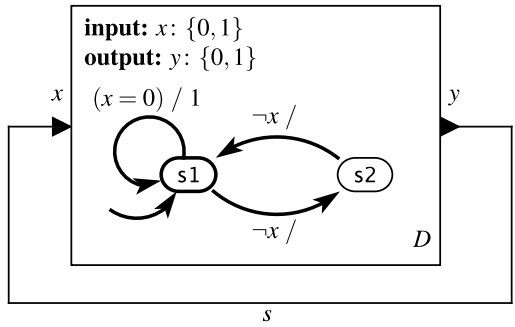


Figure 6.6: A well-formed feedback model that is not constructive.

A state machine for which the procedure works in all reachable states is said to be **constructive** (Berry, 1999). The example in Figure 6.6 is not constructive. For non-constructive machines, we are forced to do exhaustive search or to invent some more elaborate solution technique. Since exhaustive search is often too expensive for practical use, many SR languages and modeling tools (see box on page 148) reject non-constructive models.

Step 2 of the above procedure is key. How exactly can we determine the outputs if the inputs are not all known? This requires what is called a **must-may analysis** of the model. Examining the machine, we can determine what *must* be true of the outputs and what *may* be true of the outputs.

Example 6.7: The model in Figure 6.2 is constructive. In state `s1`, we can immediately determine that the machine *may not* produce an output. Therefore, we can immediately conclude that the output is *absent*, even though the input is unknown. Of course, once we have determined that the output is absent, we now know that the input is absent, and hence the procedure concludes.

In state `s2`, we can immediately determine that the machine *must* produce an output, so we can immediately conclude that the output is *present*.

The above procedure can be generalized to an arbitrary model structure. Consider for example Figure 6.1(a). There is no real need to convert it to the form of Figure 6.1(d). Instead, we can just begin by labeling all signals unknown, and then in arbitrary order, examine each actor to determine whatever can be determined about the outputs, given its initial state. We repeat this until no further progress can be made, at which point either all signals become known, or we reject the model as either ill-formed or non-constructive. Once we know all signals, then all actors can make state transitions, and we repeat the procedure in the new state for the next reaction.

The constructive procedure above can be adapted to support nondeterministic machines (see Exercise 4). But now, things become even more subtle, and there are variants to the semantics. One way to handle nondeterminism is that when executing the constructive procedure, when encountering a nondeterministic choice, make an arbitrary choice. If the result leads to a failure of the procedure to find a fixed point, then we could either reject the model (not all choices lead to a well-formed or constructive model) or reject the choice and try again.

In the SR model of computation, actors react simultaneously and instantaneously, at least conceptually. Achieving this with realistic computation requires tight coordination of the computation. We consider next a family of models of computation that require less coordination.

6.3 Dataflow Models of Computation

In this section, we consider MoCs that are much more asynchronous than SR. Reactions may occur simultaneously, or they may not. Whether they do or do not is not an essential part of the semantics. The decision as to when a reaction occurs can be much more decentralized, and can in fact reside with each individual actor. When reactions are dependent on one another, the dependence is due to the flow of data, rather than to the synchrony of events. If a reaction of actor A requires data produced by a reaction of actor B, then the reaction of A must occur after the reaction of B. A MoC where such data dependencies are the key constraints on reactions is called a **dataflow** model of computation. There are several variants of dataflow MoCs, a few of which we consider here.

Synchronous-Reactive Languages

The synchronous-reactive MoC has a history dating at least back to the mid 1980s when a suite of programming languages were developed. The term "reactive" comes from a distinction in computational systems between **transformational systems**, which accept input data, perform computation, and produce output data, and **reactive systems**, which engage in an ongoing dialog with their environment (Harel and Pnueli, 1985). Manna and Pnueli (1992) state

> "The role of a reactive program ... is not to produce a final result but to maintain some ongoing interaction with its environment."

The distinctions between transformational and reactive systems led to the development of a number of innovative programming languages. The **synchronous languages** (Benveniste and Berry, 1991) take a particular approach to the design of reactive systems, in which pieces of the program react simultaneously and instantaneously at each tick of a global clock. First among these languages are Lustre (Halbwachs et al., 1991), Esterel (Berry and Gonthier, 1992), and Signal (Le Guernic et al., 1991). Statecharts (Harel, 1987) and its implementation in Statemate (Harel et al., 1990) also have a strongly synchronous flavor.

SCADE (Berry, 2003) (Safety Critical Application Development Environment), a commercial product of Esterel Technologies, builds on Lustre, borrows concepts from Esterel, and provides a graphical syntax, where state machines are drawn and actor models are composed in a similar manner to the figures in this text. One of the main attractions of synchronous languages is their strong formal properties that yield quite effectively to formal analysis and verification techniques. For this reason, SCADE models are used in the design of safety-critical flight control software systems for commercial aircraft made by Airbus.

The principles of synchronous languages can also be used in the style of a **coordination language** rather than a programming language, as done in Ptolemy II (Edwards and Lee, 2003) and ForSyDe (Sander and Jantsch, 2004). This allows for "primitives" in a system to be complex components rather than built-in language primitives. This approach allows heterogeneous combinations of MoCs, since the complex components may themselves be given as compositions of further subcomponents under some other MoC.

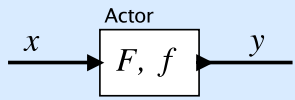
6.3.1 Dataflow Principles

In dataflow models, the signals providing communication between the actors are *sequences* of message, where each message is called a **token**. That is, a signal s is a partial function of the form

$$s: \mathbb{N} \rightharpoonup V_s,$$

where V_s is the type of the signal, and where the signal is defined on an **initial segment** $\{0, 1, \cdots, n\} \subset \mathbb{N}$, or (for infinite executions) on the entire set $\mathbb{N}$. Each element $s(n)$ of this sequence is a token. A (determinate) actor will be described as a function that maps input sequences to output sequences. We will actually use two functions, an **actor function**, which maps *entire* input sequences to *entire* output sequences, and a **firing function**, which maps a finite portion of the input sequences to output sequences, as illustrated in the following example.

Example 6.8: Consider an actor that has one input and one output port as shown below



Suppose that the input type is $V_x = \mathbb{R}$. Suppose that this is a Scale actor parameterized by a parameter $a \in \mathbb{R}$, similar to the one in Example 2.3, which multiplies inputs by a. Then

$$F(x_1, x_2, x_3, \cdots) = (ax_1, ax_2, ax_3, \cdots).$$

Suppose that when the actor fires, it performs one multiplication in the firing. Then the firing function f operates only on the first element of the input sequence, so

$$f(x_1, x_2, x_3, \cdots) = f(x_1) = (ax_1).$$

The output is a sequence of length one.

As illustrated in the previous example, the actor function F combines the effects of multiple invocations of the firing function f. Moreover, the firing function can be invoked with only partial information about the input sequence to the actor. In the above example, the firing function can be invoked if one or more tokens are available on the input. The rule requiring one token is called a **firing rule** for the Scale actor. A firing rule specifies the number of tokens required on each input port in order to fire the actor.

The Scale actor in the above example is particularly simple because the firing rule and the firing function never vary. Not all actors are so simple.

Example 6.9: Consider now a different actor Delay with parameter $d \in \mathbb{R}$. The actor function is

$$D(x_1, x_2, x_3, \cdots) = (d, x_1, x_2, x_3, \cdots).$$

This actor prepends a sequence with a token with value d. This actor has two firing functions, d_1 and d_2, and two firing rules. The first firing rule requires no input tokens at all and produces an output sequence of length one, so

$$d_1(s) = (d),$$

where s is a sequence of any length, including length zero (the empty sequence). This firing rule is initially the one used, and it is used exactly once. The second firing rule requires one input token and is used for all subsequent firings. It triggers the firing function

$$d_2(x_1, \cdots) = (x_1).$$

The actor consumes one input token and produces on its output the same token. The actor can be modeled by a state machine, as shown in Figure 6.7. In that figure, the firing rules are implicit in the guards. The tokens required to fire are exactly those required to evaluate the guards. The firing function d_1 is associated with state s1, and d_2 with s2.

When dataflow actors are composed, with an output of one going to an input of another, the communication mechanism is quite different from that of the previous MoCs consid-

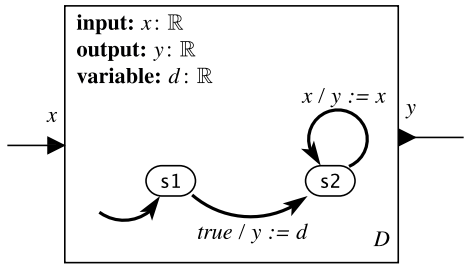
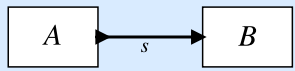


Figure 6.7: An FSM model for the Delay actor in Example 6.9.

ered in this chapter. Since the firing of the actors is asynchronous, a token sent from one actor to another must be buffered; it needs to be saved until the destination actor is ready to consume it. When the destination actor fires, it **consumes** one or more input tokens. After being consumed, a token may be discarded (meaning that the memory in which it is buffered can be reused for other purposes).

Dataflow models pose a few interesting problems. One question is how to ensure that the memory devoted to buffering of tokens is bounded. A dataflow model may be able to execute forever (or for a very long time); this is called an **unbounded execution**. For an unbounded execution, we may have to take measures to ensure that buffering of unconsumed tokens does not overflow the available memory.

Example 6.10: Consider the following cascade composition of dataflow actors:



Since A has no input ports, its firing rule is simple. It can fire at any time. Suppose that on each firing, A produces one token. What is to keep A from firing at a faster rate than B? Such faster firing could result in an unbounded build up of unconsumed tokens on the buffer between A and B. This will eventually exhaust available memory.

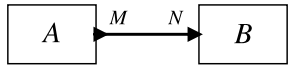


Figure 6.8: SDF actor A produces M tokens when it fires, and actor B consumes N tokens when it fires.

In general, for dataflow models that are capable of unbounded execution, we will need scheduling policies that deliver **bounded buffers**.

A second problem that may arise is **deadlock**. Deadlock occurs when there are cycles, as in Figure 6.1, and a directed loop has insufficient tokens to satisfy any of the firing rules of the actors in the loop. The Delay actor of Example 6.9 can help prevent deadlock because it is able to produce an initial output token without having any input tokens available. Dataflow models with feedback will generally require Delay actors (or something similar) in every cycle.

For general dataflow models, it can be difficult to tell whether the model will deadlock, and whether there exists an unbounded execution with bounded buffers. In fact, these two questions are undecidable, meaning that there is no algorithm that can answer the question in bounded time for all dataflow models (Buck, 1993). Fortunately, there are useful constraints that we can impose on the design of actors that make these questions decidable. We examine those constraints next.

6.3.2 Synchronous Dataflow

Synchronous dataflow (**SDF**) is a constrained form of dataflow where for each actor, every firing consumes a fixed number of input tokens on each input port and produces a fixed number of output tokens on each output port (Lee and Messerschmitt, 1987).[1]

[1]Despite the term, synchronous dataflow is not synchronous in the sense of SR. There is no global clock in SDF models, and firings of actors are asynchronous. For this reason, some authors use the term **static dataflow** rather than synchronous dataflow. This does not avoid all confusion, however, because Dennis (1974) had previously coined the term "static dataflow" to refer to dataflow graphs where buffers could hold at most one token. Since there is no way to avoid a collision of terminology, we stick with the original "synchronous dataflow" terminology used in the literature. The term SDF arose from a signal processing concept, where two signals with sample rates that are related by a rational multiple are deemed to be synchronous.

Consider a single connection between two actors, A and B, as shown in Figure 6.8. The notation here means that when A fires, it produces M tokens on its output port, and when B fires, it consumes N tokens on its input port. M and N are positive integers. Suppose that A fires q_A times and B fires q_B times. All tokens that A produces are consumed by B if and only if the following **balance equation** is satisfied,

$$q_A M = q_B N. \tag{6.2}$$

Given values q_A and q_B satisfying (6.2), we can find a schedule that delivers unbounded execution with bounded buffers. An example of such a schedule fires A repeatedly, q_A times, followed by B repeatedly, q_B times. It can repeat this sequence forever without exhausting available memory.

Example 6.11: Suppose that in Figure 6.8, $M = 2$ and $N = 3$. Then $q_A = 3$ and $q_B = 2$ satisfy (6.2). Hence, the following schedule can be repeated forever,

$$A, A, A, B, B.$$

An alternative schedule is also available,

$$A, A, B, A, B.$$

In fact, this latter schedule has an advantage over the former one in that it requires less memory. B fires as soon as there are enough tokens, rather than waiting for A to complete its entire cycle.

Another solution to (6.2) is $q_A = 6$ and $q_B = 4$. This solution includes more firings in the schedule than are strictly needed to keep the system in balance.

The equation is also satisfied by $q_A = 0$ and $q_B = 0$, but if the number of firings of actors is zero, then no useful work is done. Clearly, this is not a solution we want. Negative solutions are also not desirable.

Generally we will be interested in finding the least positive integer solution to the balance equations.

In a more complicated SDF model, every connection between actors results in a balance equation. Hence, the model defines a system of equations.

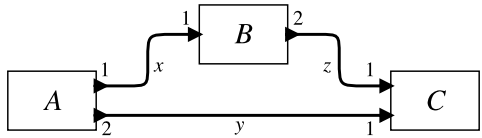


Figure 6.9: A consistent SDF model.

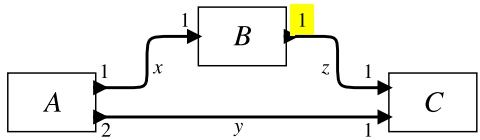


Figure 6.10: An inconsistent SDF model.

Example 6.12: Figure 6.9 shows a network with three SDF actors. The connections x, y, and z, result in the following system of balance equations,

$$\begin{aligned} q_A &= q_B \\ 2q_B &= q_C \\ 2q_A &= q_C. \end{aligned}$$

The least positive integer solution to these equations is $q_A = q_B = 1$, and $q_C = 2$, so the following schedule can be repeated forever to get an unbounded execution with bounded buffers,

$$A, B, C, C.$$

The balance equations do not always have a non-trivial solution, as illustrated in the following example.

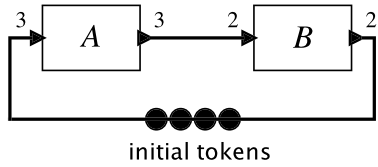


Figure 6.11: An SDF model with initial tokens on a feedback loop.

Example 6.13: Figure 6.10 shows a network with three SDF actors where the only solution to the balance equations is the trivial one, $q_A = q_B = q_C = 0$. A consequence is that there is no unbounded execution with bounded buffers for this model. It cannot be kept in balance.

An SDF model that has a non-zero solution to the balance equations is said to be **consistent**. If the only solution is zero, then it is **inconsistent**. An inconsistent model has no unbounded execution with bounded buffers.

Lee and Messerschmitt (1987) showed that if the balance equations have a non-zero solution, then they also have a solution where q_i is a positive integer for all actors i. Moreover, for connected models (where there is a communication path between any two actors), they gave a procedure for finding the least positive integer solution. Such a procedure forms the foundation for a scheduler for SDF models.

Consistency is sufficient to ensure bounded buffers, but it is not sufficient to ensure that an unbounded execution exists. In particular, when there is feedback, as in Figure 6.1, then deadlock may occur. Deadlock bounds an execution.

To allow for feedback, the SDF model treats Delay actors specially. Recall from Example 6.9, that the Delay actor is able to produce output tokens before it receives any input tokens, and then it subsequently behaves like a trivial SDF actor that copies inputs to outputs. But such a trivial actor is not really needed, and the cost of copying inputs to outputs is unnecessary. The Delay actor can be implemented very efficiently as a connection with initial tokens (those tokens that the actor is able to produce before receiving inputs). No actor is actually needed at run time. The scheduler must take the initial tokens into account.

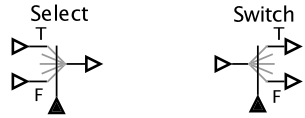


Figure 6.12: Dynamic dataflow actors.

Example 6.14: Figure 6.11 shows an SDF model with initial tokens on a feedback loop. These replace a Delay actor that is able to initially produce four tokens. The balance equations are

$$\begin{aligned} 3q_A &= 2q_B \\ 2q_B &= 3q_A. \end{aligned}$$

The least positive integer solution is $q_A = 2$, and $q_B = 3$, so the model is consistent. With four initial tokens on the feedback connection, as shown, the following schedule can be repeated forever,

$$A, B, A, B, B.$$

Were there any fewer than four initial tokens, however, the model would deadlock. If there were only three tokens, for example, then A could fire, followed by B, but in the resulting state of the buffers, neither could fire again.

In addition to the procedure for solving the balance equations, Lee and Messerschmitt (1987) gave a procedure that will either provide a schedule for an unbounded execution or will prove that no such schedule exists. Hence, both bounded buffers and deadlock are decidable for SDF models.

6.3.3 Dynamic Dataflow

Although the ability to guarantee bounded buffers and rule out deadlock is valuable, it comes at a price. SDF is not very expressive. It cannot directly express, for example, conditional firing, where an actor fires only if, for example, a token has a particular value. Such conditional firing is supported by a more general dataflow MoC known as **dynamic dataflow** (**DDF**). Unlike SDF actors, DDF actors can have multiple firing rules, and they are not constrained to produce the same number of output tokens on each firing. The Delay actor of Example 6.9 is directly supported by the DDF MoC, without any need for special treatment of initial tokens. So are two basic actors known as Switch and Select, shown in Figure 6.12.

The Select actor on the left has three firing rules. Initially, it requires one token on the bottom input port. The type of that port is Boolean, so the value of the token must be *true* or *false*. If a token with value *true* is received on that input port, then the actor produces no output, but instead activates the next firing rule, which requires one token for the top left input port, labeled *T*. When the actor next fires, it consumes the token on the *T* port and sends it to the output port. If a token with value *false* is received on the bottom input port, then the actor activates a firing rule that requires a token on the bottom left input port labeled *F*. When it consumes that token, it again sends it to the output port. Thus, it fires twice to produce one output.

The Switch actor performs a complementary function. It has only one firing rule, which requires a single token on both input ports. The token on the left input port will be sent to either the *T* or the *F* output port, depending on the Boolean value of the token received on the bottom input port. Hence, Switch and Select accomplish conditional routing of tokens, as illustrated in the following example.

Example 6.15: Figure 6.13 uses Switch and Select to accomplish conditional firing. Actor B produces a stream of Boolean-valued tokens x. This stream is replicated by the fork to provide the control inputs y and z to the Switch and Select actors. Based on the value of the control tokens on these streams, the tokens produced by actor A are sent to either C or D, and the resulting outputs are collected and sent to E. This model is the DDF equivalent of the familiar `if-then-else` programming construct in imperative languages.

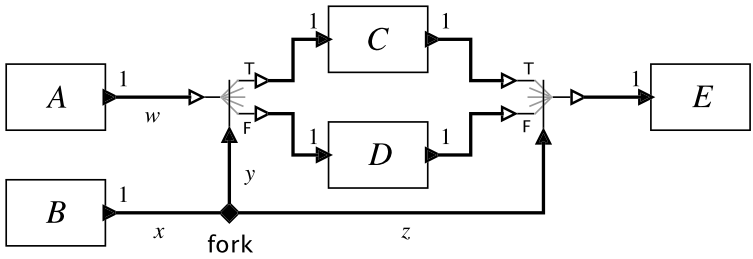


Figure 6.13: A DDF model that accomplishes conditional firing.

Addition of Switch and Select to the actor library means that we can no longer always find a bounded buffer schedule, nor can we provide assurances that the model will not deadlock. Buck (1993) showed that bounded buffers and deadlock are undecidable for DDF models. Thus, in exchange for the increased expressiveness and flexibility, we have paid a price. The models are not as readily analyzed.

Switch and Select are dataflow analogs of the **goto** statement in imperative languages. They provide low-level control over execution by conditionally routing tokens. Like goto statements, using them can result in models that are very difficult to understand. Dijkstra (1968) indicted the goto statement, discouraging its use, advocating instead the use of **structured programming**. Structured programming replaces goto statements with nested `for` loops, `if-then-else`, `do-while`, and recursion. Fortunately, structured programming is also available for dataflow models, as we discuss next.

6.3.4 Structured Dataflow

Figure 6.14 shows an alternative way to accomplish conditional firing that has many advantages over the DDF model in Figure 6.13. The grey box in the figure is an example of a **higher-order actor** called Conditional. A higher-order actor is an actor that has one or more models as parameters. In the example in the figure, Conditional is parameterized by two sub-models, one containing the actor C and the other containing the actor D. When Conditional fires, it consumes one token from each input port and produces one token on its output port, so it is an SDF actor. The action it performs when it fires, however, is dependent on the value of the token that arrives at the lower input port. If that value is true, then actor C fires. Otherwise, actor D fires.

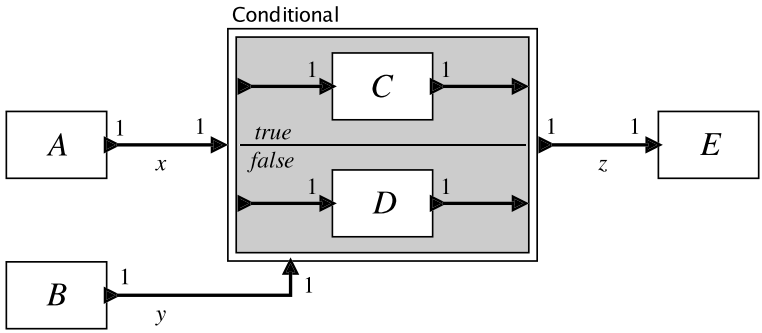


Figure 6.14: Structured dataflow approach to conditional firing.

This style of conditional firing is called **structured dataflow**, because, much like structured programming, control constructs are nested hierarchically. Arbitrary data-dependent token routing is avoided (which is analogous to avoiding arbitrary branches using goto instructions). Moreover, when using such Conditional actors, the overall model is still an SDF model. In the example in Figure 6.14, every actor consumes and produces exactly one token on every port. Hence, the model is analyzable for deadlock and bounded buffers.

This style of structured dataflow was introduced in LabVIEW, a design tool developed by National Instruments (Kodosky et al., 1991). In addition to a conditional similar to that in Figure 6.14, LabVIEW provides structured dataflow constructs for iterations (analogous to `for` and `do-while` loops in an imperative language), for `case` statements (which have an arbitrary number of conditionally executed submodels), and for sequences (which cycle through a finite set of submodels). It is also possible to support recursion using structured dataflow (Lee and Parks, 1995), but without careful constraints, boundedness again becomes undecidable.

6.3.5 Process Networks

A model of computation that is closely related to dataflow models is **Kahn process networks** (or simply, **process networks** or **PN**), named after Gilles Kahn, who introduced them (Kahn, 1974). The relationship between dataflow and PN is studied in detail by Lee and Parks (1995) and Lee and Matsikoudis (2009), but the short story is quite sim-

ple. In PN, each actor executes concurrently in its own process. That is, instead of being defined by its firing rules and firing functions, a PN actor is defined by a (typically nonterminating) program that reads data tokens from input ports and writes data tokens to output ports. All actors execute simultaneously (conceptually, whether they actually execute simultaneously or are interleaved is irrelevant).

In the original paper, Kahn (1974) gave very elegant mathematical conditions on the actors that would ensure that a network of such actors was determinate in the sense that the sequence of tokens on every connection between actors is unique, and specifically independent of how the processes are scheduled. Thus, Kahn showed that concurrent execution was possible without nondeterminacy.

Three years later, Kahn and MacQueen (1977) gave a simple, easily implemented mechanism for programs that ensures that the mathematical conditions are met to ensure determinacy. A key part of the mechanism is to perform **blocking reads** on input ports whenever a process is to read input data. Specifically, blocking reads mean that if the process chooses to access data through an input port, it issues a read request and blocks until the data becomes available. It cannot test the input port for the availability of data and then perform a conditional branch based on whether data are available, because such a branch would introduce schedule-dependent behavior.

Blocking reads are closely related to firing rules. Firing rules specify the tokens required to continue computing (with a new firing function). Similarly, a blocking read specifies a single token required to continue computing (by continuing execution of the process).

When a process writes to an output port, it performs a **nonblocking write**, meaning that the write succeeds immediately and returns. The process does not block to wait for the receiving process to be ready to receive data. This is exactly how writes to output ports work in dataflow MoCs as well. Thus, the only material difference between dataflow and PN is that with PN, the actor is not broken down into firing functions. It is designed as a continuously executing program.

Kahn and MacQueen (1977) called the processes in a PN network **coroutines** for an interesting reason. A **routine** or **subroutine** is a program fragment that is "called" by another program. The subroutine executes to completion before the calling fragment can continue executing. The interactions between processes in a PN model are more symmetric, in that there is no caller and callee. When a process performs a blocking read, it is in a sense invoking a routine in the upstream process that provides the data. Similarly, when it performs a write, it is in a sense invoking a routine in the downstream process to

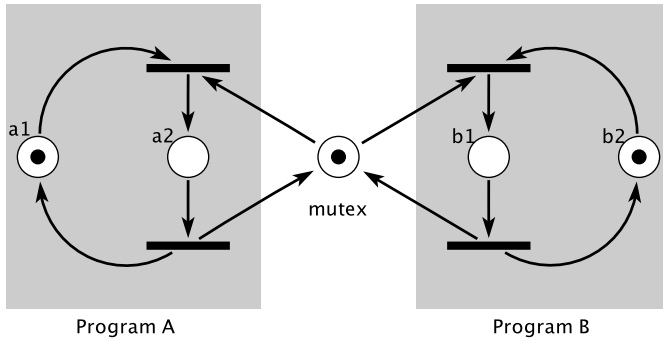


Figure 6.15: A Petri net model of two concurrent programs with a mutual exclusion protocol.

process the data. But the relationship between the producer and consumer of the data is much more symmetric than with subroutines.

Just like dataflow, the PN MoC poses challenging questions about boundedness of buffers and about deadlock. PN is expressive enough that these questions are undecidable. An elegant solution to the boundedness question is given by Parks (1995) and elaborated by Geilen and Basten (2003).

An interesting variant of process networks performs **blocking writes** rather than non-blocking writes. That is, when a process writes to an output port, it blocks until the receiving process is ready to receive the data. Such an interaction between processes is called a **rendezvous**. Rendezvous forms the basis for well known process formalisms such as **communicating sequential processes** (**CSP**) (Hoare, 1978) and the **calculus of communicating systems** (**CCS**) (Milner, 1980). It also forms the foundation for the **Occam** programming language (Galletly, 1996), which enjoyed some success for a period of time in the 1980s and 1990s for programming parallel computers.

In both the SR and dataflow models of computation considered so far, time plays a minor role. In dataflow, time plays no role. In SR, computation occurs simultaneously and instantaneously at each of a sequence of ticks of a global clock. Although the term "clock" implies that time plays a role, it actually does not. In the SR MoC, all that matters is the sequence. The physical time at which the ticks occur is irrelevant to the MoC. It is just a *sequence* of ticks. Many modeling tasks, however, require a more explicit notion of time. We examine next MoCs that have such a notion.

6.4 Timed Models of Computation

For cyber-physical systems, the time at which things occur in software can matter, because the software interacts with physical processes. In this section, we consider a few concurrent MoCs that explicitly refer to time. We describe three timed MoCs, each of which have many variants. Our treatment here is necessarily brief. A complete study of these MoCs would require a much bigger volume.

6.4.1 Time-Triggered Models

Kopetz and Grunsteidl (1994) introduced mechanisms for periodically triggering distributed computations according to a distributed clock that measures the passage of time. The result is a system architecture called a **time-triggered architecture** (**TTA**). A key contribution was to show how a TTA could tolerate certain kinds of faults, such that failures in part of the system could not disrupt the behaviors in other parts of the system (see also Kopetz (1997) and Kopetz and Bauer (2003)). Henzinger et al. (2003) lifted the key idea of TTA to the programming language level, providing a well-defined semantics for modeling distributed time-triggered systems. Since then, these techniques have come into practical use in the design of safety-critical avionics and automotive systems, becoming a key part of standards such as FlexRay, a networking standard developed by a consortium of automotive companies.

A time-triggered MoC is similar to SR in that there is a global clock that coordinates the computation. But computations take time instead of being simultaneous and instantaneous. Specifically, time-triggered MoCs associate with a computation a **logical execution time**. The inputs to the computation are provided at ticks of the global clock, but the outputs are not visible to other computations until the *next* tick of the global clock. Between ticks, there is no interaction between the computations, so concurrency difficulties such as race conditions do not exist. Since the computations are not (logically) instantaneous, there are no difficulties with feedback, and all models are constructive.

The Simulink modeling system, sold by The MathWorks, supports a time-triggered MoC, and in conjunction with another product called Real-Time Workshop, can translate such models in embedded C code. In LabVIEW, from National Instruments, timed loops accomplish a similar capability within a dataflow MoC.

In the simplest form, a time-triggered model specifies periodic computation with a fixed time interval (the **period**) between ticks of the clock. Giotto (Henzinger et al., 2003) supports modal models, where the periods differ in different modes. Some authors have further extended the concept of logical execution time to non-periodic systems (Liu and Lee, 2003; Ghosal et al., 2004).

Time triggered models are conceptually simple, but computations are tied closely to a periodic clock. The model becomes awkward when actions are not periodic. DE systems, considered next, encompass a richer set of timing behaviors.

6.4.2 Discrete Event Systems

Discrete-event systems (**DE** systems) have been used for decades as a way to build simulations for an enormous variety of applications, including for example digital networks, military systems, and economic systems. A pioneering formalism for DE models is due to Zeigler (1976), who called the formalism **DEVS**, abbreviating discrete event system specification. DEVS is an extension of Moore machines that associates a non-zero lifespan with each state, thus endowing the Moore machines with an explicit notion of the passage of time (vs. a sequence of reactions).

The key idea in a DE MoC is that events are endowed with a **time stamp**, a value in some model of time (see box on page 169). Normally, two distinct time stamps must be comparable. That is, they are either equal, or one is earlier than the other. A DE model is a network of actors where each actor reacts to input events in time-stamp order and produces output events in time-stamp order.

Example 6.16: The clock signal with period P of Example 6.1 consists of events with time stamps nP for all $n \in \mathbb{Z}$.

To execute a DE model, we can use an **event queue**, which is a list of events sorted by time stamp. The list begins empty. Each actor in the network is interrogated for any initial events it wishes to place on the event queue. These events may be destined for another actor, or they may be destined for the actor itself, in which case they will cause a reaction of the actor to occur at the appropriate time. The execution continues by selecting the

earliest event in the event queue and determining which actor should receive that event. The value of that event (if any) is presented as an input to the actor, and the actor reacts ("fires"). The reaction can produce output events, and also events that simply request a later firing of the same actor at some specified time stamp.

At this point, variants of DE MoCs behave differently. Some variants, such as DEVS, require that outputs produced by the actor have a strictly larger time stamp than that of the input just presented. From a modeling perspective, every actor imposes some non-zero delay, in that its reactions (the outputs) become visible to other actors strictly later than the inputs that triggered the reaction. Other variants permit the actor to produce output events with the same time stamp as the input. That is, they can react instantaneously. As with SR models of computation, such instantaneous reactions can create significant subtleties because inputs become simultaneous with outputs.

The subtleties introduced by simultaneous events can be resolved by treating DE as a generalization of SR (Lee and Zheng, 2007). In this variant of a DE semantics, execution proceeds as follows. Again, we use an event queue and interrogate the actors for initial events to place on the queue. We select the event from the queue with the least time stamp, and all other events with the same time stamp, present those events to actors in the model as inputs, and then fire all actors in the manner of a constructive fixed-point iteration, as normal with SR. In this variant of the semantics, any outputs produced by an actor *must* be simultaneous with the inputs (they have the same time stamp), so they participate in the fixed point. If the actor wishes to produce an output event at a later time, it does so by requesting a firing at a later time (which results in the posting of an event on the event queue).

6.4.3 Continuous-Time Systems

In Chapter 2 we consider models of continuous-time systems based on ordinary differential equations (ODEs). Specifically, we consider equations of the form

$$\dot{\mathbf{x}}(t) = f(\mathbf{x}(t), t),$$

where $\mathbf{x}: \mathbb{R} \to \mathbb{R}^n$ is a vector-valued continuous-time function. An equivalent model is an integral equation of the form

$$\mathbf{x}(t) = \mathbf{x}(0) + \int_0^t \dot{\mathbf{x}}(\tau) d\tau \tag{6.3}$$

$$= \mathbf{x}(0) + \int_0^t f(\mathbf{x}(\tau), \tau) d\tau. \tag{6.4}$$

In Chapter 2, we show that a model of a system given by such ODEs can be described as an interconnection of actors, where the communication between actors is via continuous-time signals. Equation (6.4) can be represented as the interconnection shown in Figure 6.16, which conforms to the feedback pattern of Figure 6.1(d).

Example 6.17: The feedback control system of Figure 2.3, using the helicopter model of Example 2.3, can be redrawn as shown in Figure 6.17, which conforms to the pattern of Figure 6.16. In this case, $\mathbf{x} = \dot{\theta}_y$ is a scalar-valued continuous-time function (or a vector of length one). The function f is defined as follows,

$$f(\mathbf{x}(t), t) = (K/I_{yy})(\psi(t) - \mathbf{x}(t)),$$

and the initial value of the integrator is

$$\mathbf{x}(0) = \dot{\theta}_y(0).$$

Such models, in fact, are actor compositions under a **continuous-time model of computation**, but unlike the previous MoCs, this one cannot strictly be executed on a digital computer. A digital computer cannot directly deal with the time continuum. It can, however, be approximated, often quite accurately.

The approximate execution of a continuous-time model is accomplished by a **solver**, which constructs a numerical approximation to the solution of an ODE. The study of algorithms for solvers is quite old, with the most commonly used techniques dating back to the 19th century. Here, we will consider only one of the simplest of solvers, which is known as a **forward Euler** solver.

A forward Euler solver estimates the value of $\mathbf{x}$ at time points $0, h, 2h, 3h, \cdots$, where h is called the **step size**. The integration is approximated as follows,

$$\begin{aligned}
\mathbf{x}(h) &= \mathbf{x}(0) + hf(\mathbf{x}(0), 0) \\
\mathbf{x}(2h) &= \mathbf{x}(h) + hf(\mathbf{x}(h), h) \\
\mathbf{x}(3h) &= \mathbf{x}(2h) + hf(\mathbf{x}(2h), 2h) \\
&\cdots \\
\mathbf{x}((k+1)h) &= \mathbf{x}(kh) + hf(\mathbf{x}(kh), kh).
\end{aligned}$$

This process is illustrated in Figure 6.18(a), where the "true" value of $\dot{\mathbf{x}}$ is plotted as a function of time. The true value of $\mathbf{x}(t)$ is the area under that curve between 0 and t, plus the initial value $\mathbf{x}(0)$. At the first step of the algorithm, the increment in area is approximated as the area of a rectangle of width h and height $f(\mathbf{x}(0), 0)$. This increment yields an estimate for $\mathbf{x}(h)$, which can be used to calculate $\dot{\mathbf{x}}(h) = f(\mathbf{x}(h), h)$, the height of the second rectangle. And so on.

You can see that the errors in approximation will accumulate over time. The algorithm can be improved considerably by two key techniques. First, a **variable-step solver** will vary the step size based on estimates of the error to keep the error small. Second, a more sophisticated solver will take into account the slope of the curve and use trapezoidal approximations as suggested in Figure 6.18(b). A family of such solvers known as Runge-Kutta solvers are widely used. But for our purposes here, it does not matter what solver is used. All that matters is that (a) the solver determines the step size, and (b) at each step, the solver performs some calculation to update the approximation to the integral.

When using such a solver, we can interpret the model in Figure 6.16 in a manner similar to SR and DE models. The f actor is memoryless, so it simply performs a calculation to produce an output that depends only on the input and the current time. The integrator is a state machine whose state is updated at each reaction by the solver, which uses the input to determine what the update should be. The state space of this state machine is infinite, since the state variable $\mathbf{x}(t)$ is a vector of real numbers.

Hence, a continuous-time model can be viewed as an SR model with a time step between global reactions determined by a solver (Lee and Zheng, 2007). Specifically, a continuous-time model is a network of actors, each of which is a cascade composition of a simple memoryless computation actor and a state machine, and the actor reactions are simultaneous and instantaneous. The times of the reactions are determined by a solver. The solver will typically consult the actors in determining the time step, so that for example events like level crossings (when a continuous signal crosses a threshold) can be captured precisely. Hence, despite the additional complication of having to provide a solver, the mechanisms required to achieve a continuous-time model of computation are not much different from those required to achieve SR and DE.

A popular software tool that uses a continuous-time MoC is Simulink, from The MathWorks. Simulink represents models similarly as block diagrams, which are interconnections of actors. Continuous-time models can also be simulated using the textual tool MATLAB from the same vendor. MATRIXx, from National Instruments, also supports graphical continuous-time modeling. Continuous-time models can also be integrated within

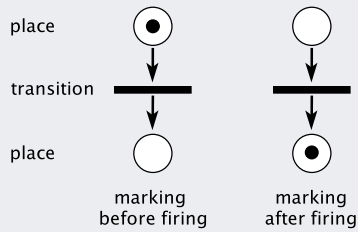
LabVIEW models, either graphically using the Control Design and Simulation Module or textually using the programming language MathScript.

6.5 Summary

This chapter provides a whirlwind tour of a rather large topic, concurrent models of computation. It begins with synchronous-reactive models, which are closest to the synchronous composition of state machines considered in the previous chapter. It then considers dataflow models, where execution can be more loosely coordinated. Only data precedences impose constraints on the order of actor computations. The chapter then concludes with a quick view of a few models of computation that explicitly include a notion of time. Such MoCs are particularly useful for modeling cyber-physical systems.

Petri Nets

Petri nets, named after Carl Adam Petri, are a popular modeling formalism related to dataflow (Murata, 1989). They have two types of elements, **places** and **transitions**, depicted as white circles and rectangles, respectively:



A place can contain any number of tokens, depicted as black circles. A transition is **enabled** if all places connected to it as inputs contain at least one token. Once a transition is enabled, it can **fire**, consuming one token from each input place and putting one token on each output place. The state of a network, called its **marking**, is the number of tokens on each place in the network. The figure above shows a simple network with its marking before and after the firing of the transition. If a place provides input to more than one transition, then the network is nondeterministic. A token on that place may trigger a firing of either destination transition.

An example of a Petri net model is shown in Figure 6.15, which models two concurrent programs with a mutual exclusion protocol. Each of the two programs has a critical section, meaning that only one of the programs can be in its critical section at any time. In the model, program A is in its critical section if there is a token on place a2, and program B is in its critical section if there is a token on place b1. The job of the mutual exclusion protocol is to ensure that these two places cannot simultaneously have a token.

If the initial marking of the model is as shown in the figure, then both top transitions are enabled, but only one can fire (there is only one token in the place labeled mutex). Which one fires is chosen nondeterministically. Suppose program A fires. After this firing, there will be a token in place a2, so the corresponding bottom transition becomes enabled. Once that transition fires, the model returns to its initial marking. It is easy to see that the mutual exclusion protocol is correct in this model.

Unlike dataflow buffers, places do not preserve an ordering of tokens. Petri nets with a finite number of markings are equivalent to FSMs.

Models of Time

How to model physical time is surprisingly subtle. How should we define simultaneity across a distributed system? A thoughtful discussion of this question is considered by Galison (2003). What does it mean for one event to cause another? Can an event that causes another be simultaneous with it? Several thoughtful essays on this topic are given in Price and Corry (2007).

In Chapter 2, we assume time is represented by a variable $t \in \mathbb{R}$ or $t \in \mathbb{R}_+$. This model is sometimes referred to as **Newtonian time**. It assumes a globally shared absolute time, where any reference anywhere to the variable t will yield the same value. This notion of time is often useful for modeling even if it does not perfectly reflect physical realities, but it has its deficiencies. Consider for example Newton's cradle, a toy with five steel balls suspended by strings. If you lift one ball and release it, it strikes the second ball, which does not move. Instead, the fifth ball reacts by rising. Consider the momentum of the middle ball as a function of time. The middle ball does not move, so its momentum must be everywhere zero. But the momentum of the first ball is somehow transfered to the fifth ball, passing through the middle ball. So the momentum cannot be always zero. Let $m: \mathbb{R} \to \mathbb{R}$ represent the momentum of this ball and τ be the time of the collision. Then

$$m(t) = \begin{cases} M & \text{if } t = \tau \\ 0 & \text{otherwise} \end{cases}$$

for all $t \in \mathbb{R}$. In a cyber-physical system, we may, however, want to represent this function in software, in which case a sequence of samples will be needed. But how can such sample unambiguously represent the rather unusual structure of this signal?

One option is to use **superdense time** (Manna and Pnueli, 1993; Maler et al., 1992; Lee and Zheng, 2005; Cataldo et al., 2006), where instead of $\mathbb{R}$, time is represented by a set $\mathbb{R} \times \mathbb{N}$. A time value is a tuple (t, n), where t represents Newtonian time and n represents a sequence index within an instant. In this representation, the momentum of the middle ball can be unambiguously represented by a sequence where $m(\tau, 0) = 0$, $m(\tau, 1) = M$, and $m(\tau, 2) = 0$. Such a representation also handles events that are simultaneous and instantaneous but also causally related.

Another alternative is **partially ordered time**, where two time values may or may not be ordered relative to each other. When there is a chain of causal relationships between them, then they must be ordered, and otherwise not.

Probing Further: Discrete Event Semantics

Discrete-event models of computation have been a subject of study for many years, with several textbooks available (Zeigler et al., 2000; Cassandras, 1993; Fishman, 2001). The subtleties in the semantics are considerable (see Lee (1999); Cataldo et al. (2006); Liu et al. (2006); Liu and Lee (2008)). Instead of discussing the formal semantics here, we describe how a DE model is executed. Such a description is, in fact, a valid way of giving the semantics of a model. The description is called an **operational semantics** (Scott and Strachey, 1971; Plotkin, 1981).

DE models are often quite large and complex, so execution performance becomes very important. Because of the use of a single event queue, parallelizing or distributing execution of DE models can be challenging (Misra, 1986; Fujimoto, 2000). A recently proposed strategy called **PTIDES** (for programming temporally integrated distributed embedded systems), leverages network time synchronization to provide efficient distributed execution (Zhao et al., 2007; Lee et al., 2009). The claim is that the execution is efficient enough that DE can be used not only as a simulation technology, but also as an implementation technology. That is, the DE event queue and execution engine become part of the deployed embedded software. As of this writing, that claim has not been proven on any practical examples.

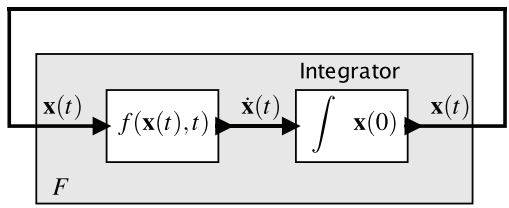


Figure 6.16: Actor model of a system described by equation (6.4).

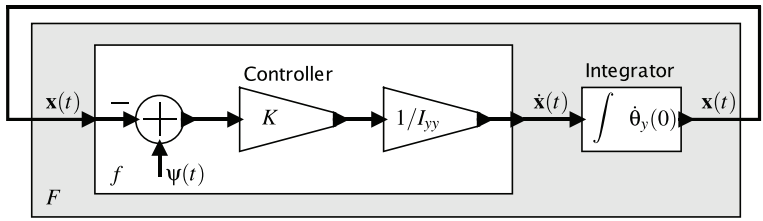


Figure 6.17: The feedback control system of Figure 2.3, using the helicopter model of Example 2.3, redrawn to conform to the pattern of Figure 6.16.

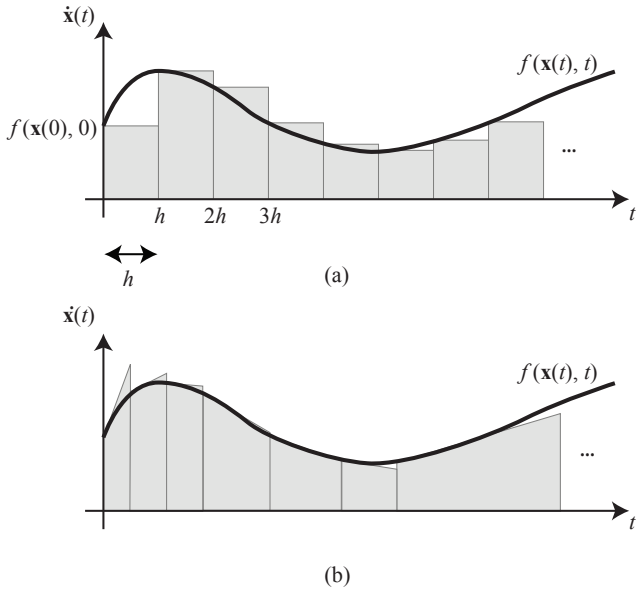
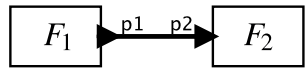


Figure 6.18: (a) Forward Euler approximation to the integration in (6.4), where **x** is assumed to be a scalar. (b) A better approximation that uses a variable step size and takes into account the slope of the curve.

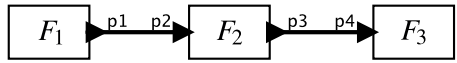
Exercises

1. Show how each of the following actor models can be transformed into a feedback system by using a reorganization similar to that in Figure 6.1(b). That is, the actors should be aggregated into a single side-by-side composite actor.

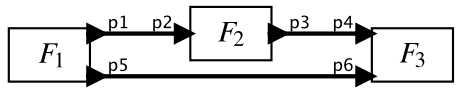
   (a)



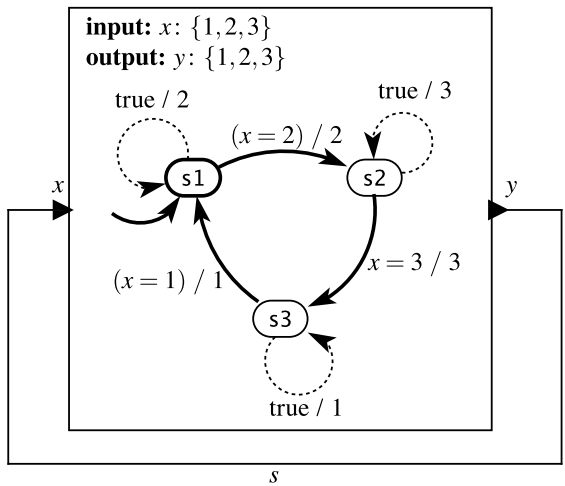
   (b)



   (c)



2. Consider the following state machine in a synchronous feedback composition:

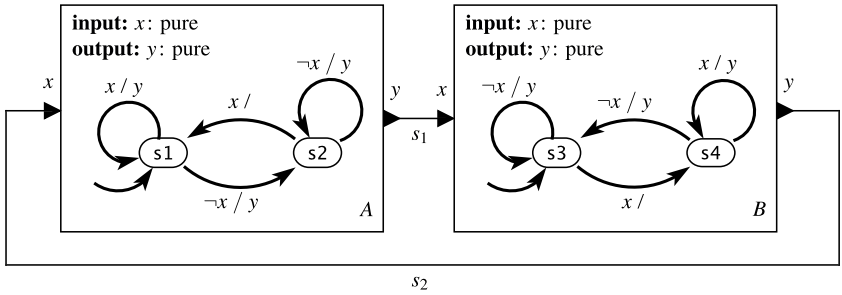


(a) Is it well-formed? Is it constructive?

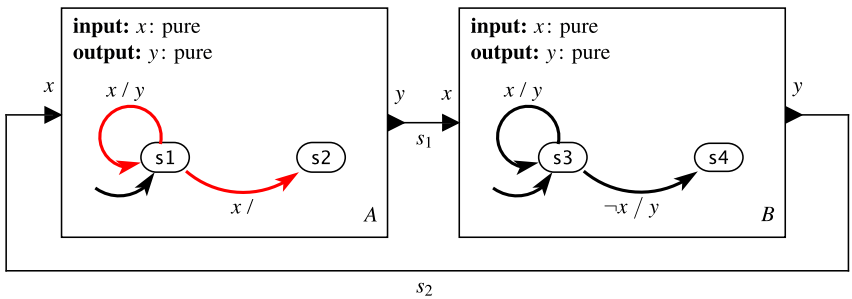
(b) If it is well-formed and constructive, then find the output symbols for the first 10 reactions. If not, explain where the problem is.

(c) Show the composition machine, assuming that the composition has no input and that the only output is y.

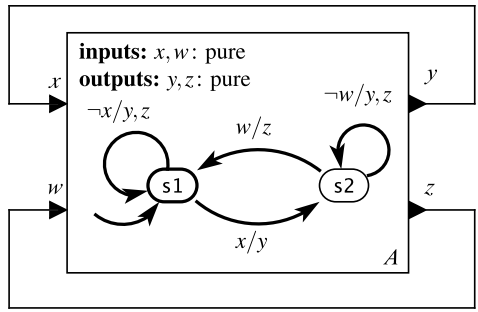
3. For the following synchronous model, determine whether it is well formed and constructive, and if so, determine the sequence of values of the signals s_1 and s_2.



4. For the following synchronous model, determine whether it is well formed and constructive, and if so, determine the possible sequences of values of the signals s_1 and s_2. Note that machine A is nondeterministic.



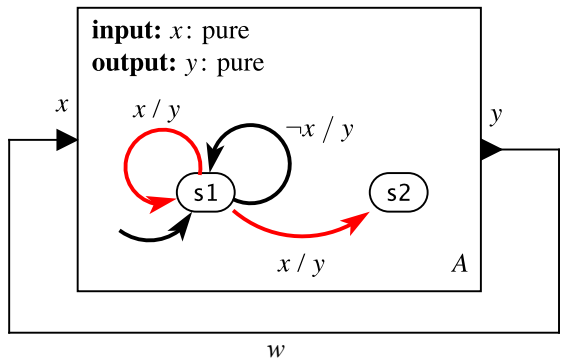
5. (a) Determine whether the following synchronous model is well formed and constructive:



Explain.

(b) For the model in part (a), give the semantics by giving an equivalent flat state machine with no inputs and two outputs.

6. Consider the following synchronous feedback composition:



Notice that the FSM A is nondeterministic.

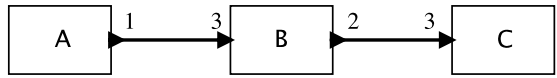
(a) Is this composition well formed? Constructive? Explain.

(b) Give an equivalent flat FSM (with no input and no connections) that produces exactly the same possible sequences w.

7. Recall the traffic light controller of Figure 3.10. Consider connecting the outputs of this controller to a pedestrian light controller, whose FSM is given in Figure 5.10. Using your favorite modeling software that supports state machines (such as Ptolemy II, LabVIEW Statecharts, or Simulink/Stateflow), construct the composition of the above two FSMs along with a deterministic extended state machine

modeling the environment and generating input symbols *timeR*, *timeG*, *timeY*, and *isCar*. For example, the environment FSM can use an internal counter to decide when to generate these symbols.

8. Consider the following SDF model:



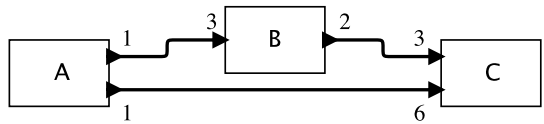
The numbers adjacent to the ports indicate the number of tokens produced or consumed by the actor when it fires. Answer the following questions about this model.

(a) Let q_A, q_B, and q_C denote the number of firings of actors A, B, and C, respectively. Write down the balance equations and find the least positive integer solution.

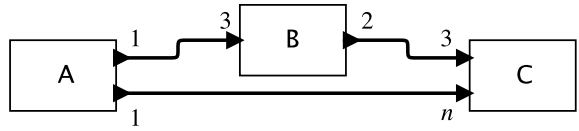
(b) Find a schedule for an unbounded execution that minimizes the buffer sizes on the two communication channels. What is the resulting size of the buffers?

9. For each of the following dataflow models, determine whether there is an unbounded execution with bounded buffers. If there is, determine the minimum buffer size.

(a)

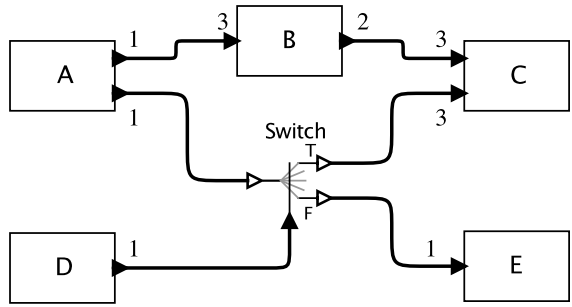


(b)



where n is some integer.

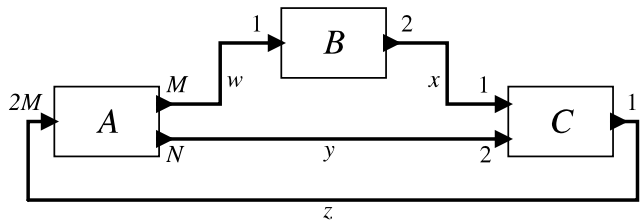
(c)



where D produces an arbitrary boolean sequence.

(d) For the same dataflow model as in part (c), assume you can specify a periodic boolean output sequence produced by D. Find such a sequence that yields bounded buffers, give a schedule that minimizes buffer sizes, and give the buffer sizes.

10. Consider the SDF graph shown below:



In this figure, A, B, and C are actors. Adjacent to each port is the number of tokens consumed or produced by a firing of the actor on that port, where N and M are variables with positive integer values. Assume the variables w, x, y, and z represent the number of initial tokens on the connection where these variables appear in the diagram. These variables have non-negative integer values.

(a) Derive a simple relationship between N and M such that the model is consistent, or show that no positive integer values of N and M yield a consistent model.

(b) Assume that $w = x = y = 0$ and that the model is consistent and find the minimum value of z (as a function N and M) such that the model does not deadlock.

(c) Assume that $z = 0$ and that the model is consistent. Find values for w, x, and y such that the model does not deadlock and $w + x + y$ is minimized.

(d) Assume that $w = x = y = 0$ and z is whatever value you found in part (b). Let b_w, b_x, b_y, and b_z be the buffer sizes for connections w, x, y, and z, respectively. What is the minimum for these buffer sizes?

Part II

Design of Embedded Systems

This part of this text studies the design of embedded systems, with emphasis on the techniques used to build concurrent, real-time embedded software. We proceed bottom up, discussing first in Chapter 7 sensors and actuators, with emphasis on how to model them. Chapter 8 covers the design of embedded processors, with emphasis on parallelism in the hardware and its implications for programmers. Chapter 9 covers memory architectures, with particular emphasis on the effect they have on program timing. Chapter 10 covers the input and output mechanisms that enable programs to interact with the external physical world, with emphasis on how to reconcile the sequential nature of software with the concurrent nature of the physical world. Chapter 11 describes mechanisms for achieving concurrency in software, threads and processes, and synchronization of concurrent software tasks, including semaphores and mutual exclusion. Finally, Chapter 12 covers scheduling, with particular emphasis on controlling timing in concurrent programs.

7 Sensors and Actuators

7.1 Models of Sensors and Actuators 181
7.1.1 Linear and Affine Models 181
7.1.2 Range 182
7.1.3 Dynamic Range 183
7.1.4 Quantization 183
7.1.5 Noise 186
7.1.6 Sampling 188
Sidebar: Decibels 189
7.1.7 Harmonic Distortion 192
7.1.8 Signal Conditioning 193
7.2 Common Sensors 195
7.2.1 Measuring Tilt and Acceleration 195
7.2.2 Measuring Position and Velocity 197
7.2.3 Measuring Rotation 199
7.2.4 Measuring Sound 199
7.2.5 Other Sensors 200
7.3 Actuators 200
7.3.1 Light-Emitting Diodes 200
7.3.2 Motor Control 202
7.4 Summary 206
Exercises 207

A **sensor** is a device that measures a physical quantity. An **actuator** is a device that alters a physical quantity. In electronic systems, sensors often produce a voltage that is proportional to the physical quantity being measured. The voltage may then be converted

to a number by an **analog-to-digital converter** (**ADC**). A sensor that is packaged with an ADC is called a **digital sensor**, whereas a sensor without an ADC is called an **analog sensor**. A digital sensor will have a limited precision, determined by the number of bits used to represent the number (this can be as few as one!). Conversely, an actuator is commonly driven by a voltage that may be converted from a number by a **digital-to-analog converter** (**DAC**). An actuator that is packaged with a DAC is called a **digital actuator**.

Today, sensors and actuators are often packaged with microprocessors and network interfaces, enabling them to appear on the Internet as services. The trend is towards a technology that deeply connects our physical world with our information world through such smart sensors and actuators. This integrated world is variously called the **Internet of Things** (**IoT**), **Industry 4.0**, the **Industrial Internet**, **Machine-to-Machine** (**M2M**), the **Internet of Everything**, the **Smarter Planet**, **TSensors** (Trillion Sensors), or **The Fog** (like The Cloud, but closer to the ground).

Some technologies for interfacing to sensors and actuators have emerged that leverage established mechanisms originally developed for ordinary Internet usage. For example, a sensor or actuator may be accessible via a web server using the so-called **Representational State Transfer** (**REST**) architectural style (Fielding and Taylor, 2002). In this style, data may be retrieved from a sensor or commands may be issued to an actuator by constructing a URL (uniform resource locator), as if you were accessing an ordinary web page from a browser, and then transmitting the URL directly to the sensor or actuator device, or to a web server that serves as an intermediary.

In this chapter, we focus not on such high-level interfaces, but rather on foundational properties of sensors and actuators as bridges between the physical and the cyber worlds. Key low-level properties include the rate at which measurements are taken or actuations are performed, the proportionality constant that relates the physical quantity to the measurement or control signal, the offset or bias, and the dynamic range. For many sensors and actuators, it is useful to model the degree to which a sensor or actuator deviates from a proportional measurement (its **nonlinearity**), and the amount of random variation introduced by the measurement process (its noise).

A key concern for sensors and actuators is that the physical world functions in a multidimensional continuum of time and space. It is an **analog** world. The world of software, however, is **digital**, and strictly quantized. Measurements of physical phenomena must be quantized in both magnitude and time before software can operate on them. And